



TEST SUITE MINIMISATION IN THE LARGE-SCALE SOFTWARE OBSERVATORIUM USING STIMULUS-RESPONSE MATRICES

Bachelor Thesis

submitted: 01.December 2025

by: Laith Philipp Sandouk

`laith.sandouk@students.uni-mannheim.de`

born May 15th, 2005

in Mannheim

Student ID Number: 1993811

Chair of Software Engineering
School of Business Informatics and Mathematics
University of Mannheim
D – 68159 Mannheim
Phone: +49 621-181-3912, Fax +49 621-181-3909
Internet: <http://swt.informatik.uni-mannheim.de>

Abstract

Large-scale industrial software systems face significant challenges in executing test suites efficiently. In a commercial SAP environment, Bach et al. report test suites comprising more than 130,000 tests requiring hundreds of hours of sequential runtime [3]. Such execution times render continuous testing computationally infeasible and economically prohibitive. The study further revealed extensive redundancy within these test suites, implying that substantial execution cost yields only marginal additional coverage benefit. However, this is not an isolated case, test-suite redundancy is a widespread and well-documented issue in industrial software development. While this computational burden cannot be eliminated entirely, its impact can be substantially reduced through test-suite minimisation, which systematically removes redundant tests with respect to defined adequacy criteria such as statement coverage, branch coverage, or mutation score [26]. Test-suite minimisation is particularly critical for regression testing, where previously passing tests are re-executed after code changes to detect newly introduced defects. However, naive reduction approaches risk compromising fault-detection effectiveness, as tests with unique defect-revealing capabilities may be inadvertently excluded.

This thesis investigates test-suite minimisation in the context of the *Large-Scale Software Observatory* (LASSO) [15], a research platform for automated selection, execution, and comparative analysis of software systems. LASSO integrates open-source repositories and combines static and dynamic analyses within a unified framework designed for language independence. Its current implementation focuses on Java and Python systems and enables empirically grounded studies of *Big Code* ecosystems (large, diverse corpora of source code enriched with meta-data such as bug-fix histories, version control traces, and execution outcomes [2]).

Our approach leverages *Stimulus–Response Matrices* (SRMs) [15], a two-dimensional representation mapping test stimuli to observed execution outcomes. We enrich SRM entries with per-test coverage information and utilise the SRM’s existing

mutation data, enabling minimisation algorithms to operate on fine-grained behavioural information rather than language-specific code structures.

Through a targeted literature survey encompassing greedy heuristics, exact optimisation, search-based, and data-driven minimisation strategies, we comparatively assessed candidate algorithms along six established criteria: SRM compatibility, coverage retention, reduction effectiveness, mutation-coverage preservation, computational scalability, and implementation feasibility. The Mutation-Aware Enhanced Harrold–Gupta–Soffa (MA-EHGS) algorithm emerged as the most promising candidate. MA-EHGS extends the empirically validated Enhanced HGS baseline [10] by integrating mutation weighting to balance coverage preservation and fault-detection power [13].

We successfully integrated MA-EHGS into LASSO and validated the implementation using SRM representations of JUnit-based test suites. The achieved reduction levels fall within the ranges reported for HGS-based and mutation-aware greedy techniques in prior empirical studies [10, 13]. Since MA-EHGS follows the same matrix-driven selection structure as other greedy test-suite minimisation algorithms, it retains the practical scalability widely observed for greedy heuristics in regression testing [26, 18].

Contents

Abstract	ii
List of Figures	viii
List of Tables	ix
List of Abbreviations	x
1. Introduction	1
1.1. Background and Motivation	1
1.2. LASSO Context	2
1.3. Research Objectives and Questions	2
1.4. Implementation Strategy	3
1.5. Thesis Structure	3
2. Fundamentals	4
2.1. Test Coverage and Mutation Score	4
2.1.1. Coverage Metrics	4
2.1.2. Mutation Score	5
2.2. Stimulus–Response Matrices	5
2.2.1. Sequence and Actuation Sheets	5
2.2.2. From Sequence Sheets to SRMs	6
2.3. The Test-Suite Minimisation Problem	7
2.3.1. Concept and Objectives	7
2.3.2. Computational Complexity and Trade-offs	7
2.3.3. Minimisation within LASSO	8
2.3.4. Algorithmic Categories	8
3. Literature Review	9
3.1. Greedy Heuristics	9
3.1.1. Classical Greedy	9

3.1.2.	Harrold–Gupta–Soffa (HGS)	10
3.1.3.	Delayed Greedy (DG)	10
3.1.4.	Enhanced HGS (EHGS)	11
3.1.5.	Mutation-Aware Enhanced HGS (MA-EHGS)	12
3.2.	Exact Optimisation Approaches	12
3.2.1.	ILP, SAT, and Set-Cover Formulations	13
3.2.2.	Nonlinear Multi-Criteria Optimisation (NEMO)	13
3.2.3.	Structure-Aware Optimisation (REDUNET)	13
3.3.	Search-Based and Metaheuristic Approaches	14
3.3.1.	Genetic Algorithms	14
3.3.2.	Other Metaheuristic Approaches	15
3.4.	Data-Driven and Similarity-Based Approaches	15
3.4.1.	Similarity-Based and Time-Constrained Methods	15
3.5.	Summary	16
4.	Evaluation Criteria and Scoring Framework	17
4.1.	Evaluation Criteria (C1–C6)	17
4.1.1.	Mandatory Precondition	17
4.1.2.	Performance Criteria	17
4.2.	Application to Algorithm Comparison	18
5.	Comparative Evaluation and Selection	19
5.1.	Criterion C1: SRM Compatibility	19
5.2.	Criterion C2: Coverage Preservation	20
5.3.	Criterion C3: Reduction Effectiveness	20
5.4.	Criterion C4: Mutation Score Preservation	21
5.5.	Criterion C5: Computational Scalability	21
5.6.	Criterion C6: Implementation Feasibility	22
5.7.	Selection Decision	22
6.	Implementation	24
6.1.	SRM Data Collection	24
6.1.1.	Extending Arena for Per-Test Coverage	24
6.1.2.	Collecting Per-Test Mutation Scores	25
6.2.	Algorithm Implementation	26
6.2.1.	Phase 1: Essential Test Selection	26

Contents	vi
6.2.2. Phase 2: Weighted Greedy Selection	26
6.2.3. Complexity Analysis	27
6.3. LSL Action Integration	27
6.3.1. Configurable Parameters	27
6.3.2. Execution Workflow	28
6.4. Parameter Sensitivity Validation	28
7. Discussion	29
7.1. General Advantages and Disadvantages of Test Suite Minimisation	29
7.1.1. Practical Performance Benefits	29
7.1.2. The Absence of a Perfect Solution	30
7.1.3. The Coverage Limitation	30
7.2. Thesis-Specific Considerations for MA-EHGS in LASSO	31
7.2.1. Advantages of the Approach	31
7.2.2. Limitations of the Implementation	31
7.2.3. Implications for LASSO Platform Evolution	32
7.3. Future Work	33
8. Conclusion	34
8.1. Summary	34
8.2. Key Contributions	34
8.3. Future Work	35
Bibliography	vii
Appendix	xi
Detailed Evaluation Tables for Comparative Analysis	xii
A.1. Scoring Methodology	xii
B.2. C1: SRM Compatibility	xiv
C.3. C2: Coverage Preservation	xv
D.4. C3: Reduction Effectiveness	xvi
E.5. C4: Mutation Score Preservation	xviii
F.6. C5: Computational Scalability	xix
G.7. C6: Implementation Feasibility	xxi
H.8. Complete Multi-Criteria Scoring Table	xxii

I.9. MA-EHGS Selection Justification	xxii
Mutation-Aware Enhanced HGS Pseudocode	xxiv
Extended Benchmarks and Reproducibility	xxviii
SRM Coverage Extraction Specification	xxxi
Licensing Header Template	xxxiii

List of Figures

2.1.	Relationship between a sequence sheet (stimulus) and its corresponding actuation sheet (observed behaviour).	6
2.2.	Example SRM fragment showing actual outputs across different system variants (columns). Green cells indicate passing tests; red cells indicate failures. Systems 1 and 3 produce identical output (42) for Test 1, while Systems 2 and 4 produce different mutant outputs (41, 43). All systems pass Test 2 with identical output (100).	6
2.3.	The test-suite minimisation problem formulated as an optimisation balancing suite size and coverage retention.	8
6.1.	Example of mutation kill detection. Red cells indicate mutant kills.	25

List of Tables

1.	C1: SRM Compatibility—mandatory precondition (pass/fail). . .	xiv
2.	C2: Structural Coverage Preservation (0/1).	xv
3.	C3: Reduction Effectiveness—achieves $\geq 45\%$ reduction (0/1). . .	xvi
4.	C4: Mutation Score Preservation—explicit mutation awareness (0/1).	xviii
5.	C5: Computational Scalability—polynomial-time (0/1).	xix
6.	C6: Implementation Feasibility—simple, matrix-based, no external dependencies (0/1).	xxi
7.	Multi-criteria binary scores (C2–C6, maximum 5/5).	xxii
1.	MA-EHGS validation: parameter sensitivity and cross-implementation consistency.	xxix

List of Abbreviations

ABC		Artificial Bee Colony
AST		Abstract Syntax Tree
CFG		Control Flow Graph
CSP		Constraint Satisfaction Problem
DG		Delayed Greedy
EHGS		Enhanced HGS
GA		Genetic Algorithm
GE		Greedy Effective
GRE		Greedy Redundant Eliminating
HGS		Harrold–Gupta–Soffa algorithm
ILP		Integer Linear Programming
LASSO		Large-Scale Software Observatory
LSL		LASSO Scripting Language
MA-EHGS		Mutation-Aware Enhanced HGS
MOEA		Many-Objective Evolutionary Algorithm
NEMO		Nonlinear Multi-criteria Integer Programming
NSGA-II		Non-dominated Sorting Genetic Algorithm II
PSO		Particle Swarm Optimisation
SAT		Boolean Satisfiability Problem
SRM		Stimulus–Response Matrix
SSR		Suite Size Reduction

1. Introduction

Software testing ensures that program modifications do not introduce defects. As systems evolve, test suites grow and increase execution cost and maintenance effort. Modern suites often combine human-written, automatically generated, and in recent years more frequently AI-generated tests. Tools such as Randoop [20] and EvoSuite [8] can produce thousands of tests, and large industrial suites may require days or weeks to execute [22]. Although such suites achieve high coverage and mutation scores, they frequently contain redundant tests whose requirements are subsumed by others, resulting in prolonged execution, increased costs, and elevated maintenance overhead.

Test-suite minimisation addresses this issue by systematically removing redundant tests while preserving fault-detection capability [26]. Techniques range from *coverage-preserving* approaches, which maintain all coverage requirements, to *coverage-relaxing* approaches, which accept limited coverage loss for stronger reduction. While minimisation can significantly reduce computational demand and feedback latency, overly aggressive reduction risks discarding tests with unique defect-revealing behaviour. Effective minimisation therefore requires balancing reduction efficiency with fault-detection retention.

1.1. Background and Motivation

Testing remains a major cost driver in software engineering. Classical studies estimate that verification and validation account for up to half of total development effort [4]. The cost pressure persists in modern continuous-integration environments, where automated or AI-generated suites may contain tens of thousands of tests [22, 3]. Executing all tests after each change is often infeasible, motivating techniques that reduce suite size while retaining quality. Test suite minimisation has been exploring this issue for decades, with numerous algorithms proposed to

identify and eliminate redundant tests [26], by leveraging coverage and quite recently mutation information as a proxy for fault-detection capability [13]. Details on coverage and mutation testing are provided in Chapter 2.

1.2. LASSO Context

The *Large-Scale Software Observatory* (LASSO) [15] is a research platform for scalable analysis of large software corpora (*Big Code*). LASSO automates the extraction of static properties and dynamic execution behaviour from open-source systems and enables reproducible pipelines through the *LASSO Scripting Language* (LSL). This makes it a central infrastructure for empirical software engineering and for evaluating emerging techniques such as AI-assisted code generation.

LASSO represents system behaviour through *Stimulus–Response Matrices* (SRMs) [15], where rows denote test stimuli, columns denote systems, and cells contain structured response objects. SRMs allow language-independent extraction of coverage metrics, mutation scores, and behavioural characteristics, enabling systematic comparison of functionally equivalent implementations. Section 2.2 provides formal definitions.

Despite supporting large-scale behavioural and coverage analysis, LASSO currently lacks integrated test-suite minimisation. Redundant test executions accumulate significant computational overhead across the platform. Integrating minimisation would reduce this overhead while preserving the behavioural coverage required for reliable system analysis.

1.3. Research Objectives and Questions

This thesis investigates how test-suite minimisation can be integrated into LASSO using its SRM abstraction. The study is guided by:

1. **RQ1:** Which minimisation techniques demonstrate the highest practical effectiveness in the literature for the LASSO platform?
2. **RQ2:** Which evaluation criteria best suit minimisation within LASSO’s SRM-based environment?

3. **RQ3:** Which approach best balances reduction effectiveness, coverage preservation, mutation preservation and scalability?
4. **RQ4:** How can the selected approach be integrated into LASSO while maintaining language independence?

1.4. Implementation Strategy

This thesis performs minimisation directly on SRMs rather than relying on external tools, which typically require source-code access or language-specific instrumentation. SRMs already encapsulate all data required for behavioural minimisation (test identifiers, execution results, coverage information, and mutation data), allowing language-independent and scalable processing while ensuring seamless integration into LASSO’s analysis pipeline.

1.5. Thesis Structure

The remainder of this thesis is organised as follows:

Chapter 2 introduces coverage, mutation testing, SRMs, and formally defines the minimisation problem.

Chapter 3 surveys greedy, exact, search-based, and data-driven minimisation techniques.

Chapter 4 defines evaluation criteria (C1–C6) and the scoring framework.

Chapter 5 applies the framework to select the Mutation-Aware Enhanced HGS approach.

Chapter 6 describes the implementation and integration into LASSO.

Chapter 7 discusses limitations, trade-offs, and deployment considerations.

Chapter 8 concludes and outlines future work.

2. Fundamentals

This chapter establishes the theoretical foundation for this work. It introduces the core concepts of test coverage and mutation testing, explains the *Stimulus–Response Matrix* (SRM) representation used within LASSO, and formalises the test-suite minimisation problem, including trade-offs between coverage preservation and reduction strength.

2.1. Test Coverage and Mutation Score

Test coverage quantifies the proportion of program elements exercised by a test suite [27]. Coverage metrics indicate structural thoroughness but provide limited insight into fault-detection capability.

2.1.1. Coverage Metrics

Coverage criteria define program ”parts” at different granularity levels [27]. While numerous coverage types exist (e.g., instruction coverage, path coverage, class coverage), this work focuses on the three metrics most commonly reported in test-suite minimisation literature [26, 11, 13, 18]:

- **Line Coverage:** proportion of source lines executed.
- **Branch Coverage:** proportion of conditional branches whose true/false outcomes are both executed.
- **Method Coverage:** proportion of methods invoked at least once.

These three metrics capture execution at different granularities and are widely supported by standard coverage tools such as JaCoCo. Combining multiple metrics provides a more complete assessment of test quality, as high line coverage with low branch coverage may miss conditional faults.

2.1.2. Mutation Score

Mutation testing measures test effectiveness by introducing small syntactic changes (*mutants*) into the program and observing whether tests detect the injected faults [14]. The *mutation score* is defined as:

$$\text{Mutation Score} = \frac{\text{Number of mutants killed}}{\text{Total number of mutants}} \times 100 \, \%.$$

A mutant is *killed* if the test causes observable behavioural divergence from the original program.

Mutation Types. Two principal mutation styles are distinguished [19]:

1. **Weak Mutation:** detects state deviations immediately after a mutated statement executes.
2. **Strong Mutation:** detects externally visible behavioural differences at program output.

Weak mutation captures local sensitivity to code changes. Strong mutation quantifies global behavioural divergence. Mutation scores thereby approximate a suite’s fault-detection capability.

2.2. Stimulus–Response Matrices

Stimulus–Response Matrices (SRMs) constitute LASSO’s central data structure for *Test-Driven Software Experimentation* [15]. They provide a unified, language-independent format for recording and comparing run-time behaviour under controlled stimuli.

2.2.1. Sequence and Actuation Sheets

Sequence Sheets describe ordered stimuli applied to systems under test. Each row represents one method invocation with inputs and expected outputs. **Actuation Sheets** record the corresponding observed outputs and other run-time data during execution.

Stimulus (Sequence Sheet)				Observed Behaviour (Actuation Sheet)			
Out	Operation	Service	Input	Out	Operation	Service	Input
create	create	'Stack					
–	push	A1	42	<instance>	create	'Stack	
42	pop	A1		true	push	A1	42
				42	pop	A1	

Figure 2.1.: Relationship between a sequence sheet (stimulus) and its corresponding actuation sheet (observed behaviour).

2.2.2. From Sequence Sheets to SRMs

LASSO generalises stimulus-to-actuation mappings into multidimensional *Stimulus–Response Matrices* (SRMs):

$$M = [M_{ij}], \quad M_{ij} = \text{Actuation}(t_i, v_j),$$

where each row t_i represents a stimulus (test), each column v_j a system variant, and each cell M_{ij} stores execution outcomes and associated analysis data such as coverage and mutation attributes.

	System 1	System 2	System 3	System 4
Test 1	42	41	42	43
Test 2	100	100	100	100
Test 3	true	true	false	true

Figure 2.2.: Example SRM fragment showing actual outputs across different system variants (columns). Green cells indicate passing tests; red cells indicate failures. Systems 1 and 3 produce identical output (42) for Test 1, while Systems 2 and 4 produce different mutant outputs (41, 43). All systems pass Test 2 with identical output (100).

SRMs unify functional results, coverage data, and mutation outcomes in a single matrix, enabling language-independent behavioural analysis.

2.3. The Test-Suite Minimisation Problem

2.3.1. Concept and Objectives

Test-suite minimisation seeks to remove redundant tests from an existing suite while maintaining sufficient coverage and fault-detection capability [26]. Given a suite $T = \{t_1, \dots, t_n\}$ and a set of coverage requirements $R = \{r_1, \dots, r_m\}$, each test t_i covers a subset $C(t_i) \subseteq R$. The objective is to find a smaller subset $S^* \subseteq T$ that provides comparable or identical coverage.

Two major formulations exist:

- **Coverage-preserving minimisation:** guarantees full coverage, $C(S^*) = C(T)$. It ensures no requirement is lost but often limits reduction potential.
- **Coverage-relaxing minimisation:** allows limited loss of coverage, $C(S^*) \subset C(T)$, to achieve greater reduction. This variant trades fault-detection certainty for smaller suite size.

Both forms address the same optimisation tension: *minimise suite size while retaining sufficient adequacy*. The general optimisation can be expressed as

$$S^* = \arg \min_{S \subseteq T} |S| \quad \text{subject to} \quad |C(S)| \geq \gamma \cdot |C(T)|,$$

where $0 < \gamma \leq 1$ controls the acceptable coverage retention threshold ($\gamma = 1$ for coverage-preserving, $\gamma < 1$ for coverage-relaxing).

2.3.2. Computational Complexity and Trade-offs

The coverage-preserving variant corresponds to the *minimum set-cover problem*, which is NP-complete [26]. No polynomial-time algorithm guarantees optimality for arbitrary inputs, and practical implementations rely on heuristics or approximations [11, 6]. Some greedy heuristics achieve linear complexity $O(nm)$ in practice, balancing scalability and quality.

Coverage-relaxing algorithms may yield stronger reductions but risk discarding unique defect-revealing tests, potentially lowering mutation scores or missing rare execution paths. Hence, every minimisation approach must balance the trade-off between computational efficiency, reduction rate, and fault-detection retention.

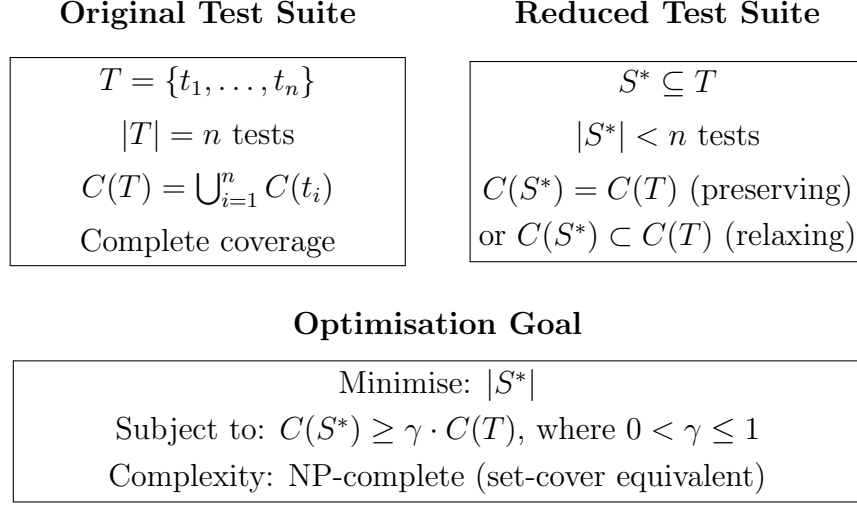


Figure 2.3.: The test-suite minimisation problem formulated as an optimisation balancing suite size and coverage retention.

2.3.3. Minimisation within LASSO

In LASSO, each test corresponds to one SRM row. Coverage and mutation data stored in SRM cells provide the requirement mappings $C(t_i)$. Minimisation thus operates directly on SRMs by selecting a subset of rows S^* that preserves coverage and mutation adequacy across all variants:

$$C(S^*)_{\text{SRM}} \geq \gamma \cdot C(T)_{\text{SRM}}.$$

This SRM-based formulation supports both coverage-preserving and coverage-relaxing strategies while remaining language-independent, enabling scalable experimentation across diverse software systems.

2.3.4. Algorithmic Categories

Following established classifications [26, 21, 3], algorithms group into four categories: **greedy heuristics** (classical greedy, HGS [11]), **exact optimisation** (ILP, constraint-satisfaction [17]), **search-based methods** (genetic algorithms [26]), **data-driven methods** (clustering, overlap-aware [3]). Detailed explanation in Chapter 3.

3. Literature Review

This chapter surveys test-suite minimisation algorithms across four main families: greedy heuristics, exact optimisation approaches, search-based (metaheuristic) techniques, and data-driven similarity-based methods. The focus is on how these approaches formulate the minimisation problem, which information they use (e.g. coverage matrices, mutants, control-flow graphs), and how they construct reduced test suites. Comparative scoring and ranking are deferred to Chapter 5.

3.1. Greedy Heuristics

Greedy heuristics are among the most widely studied techniques for test-suite minimisation [26, 18]. They operate directly on a test–requirement incidence matrix (often a coverage matrix) and construct a reduced suite incrementally: in each step, the algorithm selects a test that contributes beneficial coverage for currently uncovered requirements, until all requirements are covered. Because of their simplicity, low implementation overhead, and good practical performance, greedy algorithms are frequently used as baselines in empirical studies.

3.1.1. Classical Greedy

The classical greedy heuristic repeatedly selects the test that covers the largest number of yet-uncovered requirements. Chvátal showed that this algorithm achieves a logarithmic approximation bound for the set-cover problem [6]. Rothermel et al. and later surveys treat classical greedy as a standard baseline for regression test-suite minimisation [22, 26].

In a typical formulation, the algorithm maintains a set of uncovered requirements and, in each iteration, scores every remaining test by the number of uncovered requirements it satisfies. The test with the highest score is added to the reduced suite, and its covered requirements are marked as covered. This continues until no

uncovered requirements remain. The algorithm operates purely on the coverage matrix, without additional program structure.

Noemmer and Haas observe that such greedy variants are “the test suite minimization algorithms most commonly found in research” [18]. In their Java study, they investigate both classical greedy and more advanced greedy heuristics with respect to structural coverage and mutation testing. Jehan and Wotawa report similar results for JavaScript projects and use classical greedy and related variants as central baselines in their empirical analysis [13].

3.1.2. Harrold–Gupta–Soffa (HGS)

Harrold, Gupta and Soffa proposed the Harrold–Gupta–Soffa (HGS) algorithm as one of the early coverage-driven minimisation heuristics [11]. HGS extends the basic greedy idea by taking into account the “difficulty” of covering certain requirements. Roughly, the algorithm proceeds in phases:

- Tests are grouped according to the requirements they cover.
- Requirements that are covered by only a few tests are given priority.
- At each step, HGS selects tests that help cover such “hard-to-cover” requirements while avoiding redundant tests whose coverage is already subsumed by others.

Harrold et al. derive a worst-case time bound of $O(n(n+m)MAXCARD)$, where n denotes the number of testing sets, m the number of test cases, and $MAXCARD$ the maximum number of requirements covered by any single test [11]. HGS has since become a standard greedy baseline and appears in many comparative studies of test-suite minimisation [26, 5, 18].

Chen and Lau introduce two related greedy variants, Greedy Effective (GE) and Greedy Redundant Eliminating (GRE), which modify how tests are selected and how redundant tests are removed while maintaining the same coverage-matrix abstraction [5].

3.1.3. Delayed Greedy (DG)

Tallam and Gupta propose the Delayed Greedy (DG) algorithm to address a limitation of classical greedy and HGS: if very large tests are selected early, they

may dominate the solution and render other tests redundant, even when those smaller tests might later have formed part of a more compact suite [24].

DG uses formal concept analysis to identify and remove subsumed tests before the greedy selection phase. Conceptually, the algorithm:

- Builds a concept lattice over tests and requirements, where each concept represents a set of tests with a common set of covered requirements;
- Eliminates tests whose coverage is entirely contained in the coverage of other tests or combinations of tests;
- Applies a greedy selection strategy on the reduced set of tests.

This two-step design aims to reduce redundancy before greedy selection, potentially leading to smaller resulting suites. DG still operates on the test–requirement matrix, but the concept lattice construction incurs additional computational and implementation overhead compared with simpler greedy methods [24].

3.1.4. Enhanced HGS (EHGS)

Gladston and Ramesh refine HGS and evaluate several variants on Java programs, introducing the *Enhanced HGS* (EHGS) algorithm [10]. EHGS retains the basic HGS structure but emphasises two ideas:

1. **Unique coverage first:** EHGS first selects tests that are the only ones to cover certain requirements (so-called essential tests).
2. **Residual coverage next:** In subsequent steps, EHGS ranks remaining tests by how many still-uncovered requirements they satisfy and selects tests in decreasing order of this residual coverage.

The first phase ensures that no requirement uniquely associated with a single test is accidentally lost. The second phase then applies a coverage-based greedy strategy restricted to the remaining requirements. Gladston and Ramesh evaluate EHGS and related variants empirically and show that EHGS yields competitive suite sizes in their benchmarks [10].

EHGS operates entirely on a test–requirement matrix and does not depend on language-specific control-flow information. This makes it a natural candidate for SRM-based settings where coverage data are available in matrix form.

3.1.5. Mutation-Aware Enhanced HGS (MA-EHGS)

The Mutation-Aware Enhanced HGS (MA-EHGS) algorithm, proposed in this thesis, extends EHGS to explicitly incorporate mutation information. Recent empirical studies have shown that purely coverage-based minimisation can reduce mutation score, especially when reduction is aggressive [18, 13]. At the same time, many-objective evolutionary algorithms demonstrate that treating mutation score as an explicit optimisation objective can help preserve fault detection while still reducing cost [25]. This evolution in the literature reflects a broader shift from purely coverage-driven adequacy towards fault-based adequacy metrics.

MA-EHGS builds on this development by embedding mutation awareness into a deterministic, greedy, SRM-based heuristic. It retains the two-phase structure of EHGS (essential test selection followed by greedy residual coverage) but extends the requirements matrix to include both structural coverage (lines, branches, methods) and mutation requirements (killed mutants). A weighted scoring function balances structural and mutation contributions during test selection:

$$\text{score}(t_i) = \alpha \cdot |\text{newCoverage}_i| + \beta \cdot |\text{newMutants}_i|,$$

where α and β control the relative importance of each dimension.

This design bridges greedy coverage-based minimisation [26, 18] and mutation-aware optimisation [13, 25], operating entirely on the SRM representation without requiring program-structure analysis. A detailed description of the algorithm’s implementation, data collection pipeline, and integration within LASSO is provided in Chapter 6.

3.2. Exact Optimisation Approaches

Exact optimisation approaches treat test-suite minimisation as a formal optimisation problem, typically formulated as Integer Linear Programming (ILP), Boolean Satisfiability (SAT), or related constraint models. These methods aim to compute an optimal solution according to a given objective, but inherit the NP-hardness of the underlying hitting-set formulation [9].

3.2.1. ILP, SAT, and Set-Cover Formulations

In ILP formulations, each test is associated with a binary decision variable indicating whether it is selected, and each requirement induces a constraint ensuring that at least one selected test covers it [26, 7]. The objective function typically minimises the number of selected tests or a weighted cost reflecting test execution time.

SAT and Constraint Satisfaction Problem (CSP) encodings follow a similar idea, but express constraints in propositional logic or more general constraint languages [12]. The resulting formulas are then solved using off-the-shelf SAT or CSP solvers, providing a precise link between test-suite minimisation and classical optimisation technology.

While worst-case complexity remains exponential, ILP and SAT can solve small-to medium-sized instances and are often used to provide optimality baselines for evaluating heuristics [26, 21].

3.2.2. Nonlinear Multi-Criteria Optimisation (NEMO)

Lin et al. argue that linear models may not capture interactions between test cases, especially when multiple criteria such as cost and fault detection need to be considered simultaneously [16]. Their Nonlinear Multi-criteria Integer Programming (NEMO) framework extends the linear ILP formulation with nonlinear terms to model such interdependencies.

NEMO allows multiple objectives, such as maximising fault detection while minimising execution cost, and encodes them as a single nonlinear integer program. This approach demonstrates how richer optimisation models can be used to express complex adequacy notions. However, the increased model expressiveness comes at the price of higher computational effort compared with linear ILP [16].

3.2.3. Structure-Aware Optimisation (REDUNET)

Mongiovi et al. propose REDUNET, which combines set-cover reasoning with optimisation over Control Flow Graphs (CFGs) [17]. Instead of working solely with a coverage matrix, REDUNET analyses control-flow and data-flow dependencies

in the program to identify redundant tests. It then uses ILP-based reasoning over this CFG-enriched representation to select a reduced suite.

REDUNET illustrates a different dimension in the design space: leveraging program structure (CFGs) to guide test reduction. This approach can exploit structural information beyond what is available in a pure SRM setting, but it also requires access to language-specific program representations and associated analysis tools.

3.3. Search-Based and Metaheuristic Approaches

Search-based approaches treat test-suite minimisation as an optimisation problem and use metaheuristics such as Genetic Algorithms (GAs) or swarm-intelligence methods to explore the space of candidate subsets [26, 21]. Instead of constructing a single solution greedily, these techniques maintain a population of candidate solutions and iteratively improve them via stochastic operators.

3.3.1. Genetic Algorithms

Genetic Algorithms represent each candidate test suite as a chromosome, often a binary vector indicating which tests are selected [26]. A typical GA for test-suite minimisation proceeds as follows:

- Initialise a population of random or heuristically constructed test subsets;
- Evaluate each candidate using a fitness function, such as coverage achieved minus cost, or a multi-objective fitness vector;
- Select fitter candidates for reproduction;
- Apply crossover (combining parts of two parents) and mutation (randomly flipping selection bits) to generate new candidates;
- Repeat evaluation and selection over multiple generations.

GAs can accommodate complex objectives and constraints, but require choices for population size, number of generations, crossover and mutation rates, and other parameters [26, 21]. Many GA-based minimisation approaches still optimise primarily for coverage and cost, with mutation information incorporated less frequently [18].

3.3.2. Other Metaheuristic Approaches

Beyond GAs, several other metaheuristics have been applied to test-suite minimisation. Ahuja and Bhatia evaluate Particle Swarm Optimisation (PSO) and Artificial Bee Colony (ABC) algorithms, where each particle or bee represents a candidate suite and moves through the search space guided by heuristic rules [1]. These methods follow a similar pattern: they maintain a population of candidate solutions and iteratively update them using algorithm-specific operators.

Wei et al. explore Many-Objective Evolutionary Algorithms (MOEAs) such as Non-dominated Sorting Genetic Algorithm II (NSGA-II), MOEA/D, IBEA, and SPEA2-SDE, with multiple objectives including mutation score [25]. By treating mutation score as one objective among several, MOEAs can generate trade-off fronts between cost, coverage, and mutation preservation. This line of work demonstrates that mutation-aware optimisation is feasible within population-based search and provides additional evidence that mutation score is an important adequacy dimension alongside structural coverage.

3.4. Data-Driven and Similarity-Based Approaches

Data-driven and similarity-based approaches use information about coverage patterns to identify redundant or highly overlapping tests [26, 21]. Instead of solving a hitting-set formulation directly, they analyse the structure of coverage vectors and cluster or prioritise tests based on similarity.

3.4.1. Similarity-Based and Time-Constrained Methods

Similarity-based methods typically represent each test as a high-dimensional coverage vector and apply distance or similarity measures (e.g. Jaccard similarity) to identify groups of tests that cover similar requirements. Tests within the same cluster are candidates for reduction or reordering.

Bach et al. apply such techniques in a large industrial environment at SAP, where test execution is subject to strict time budgets [3]. Their methods penalise tests whose coverage overlaps strongly with other tests and prioritise those that provide

new or diverse coverage. The goal is to maximise coverage within a given time budget rather than to compute a minimal hitting set.

Khan et al. survey similarity- and clustering-based methods and emphasise their flexibility for prioritisation and selection tasks [21]. However, because these methods focus on coverage overlap and time constraints, they are typically used for test-case prioritisation and selection under resource limits rather than for exact minimisation in the classical sense.

3.5. Summary

The reviewed literature reveals four main families of test-suite minimisation approaches: greedy heuristics (Classical Greedy, HGS, EHGS, DG) offer simplicity and scalability but traditionally focus only on structural coverage [26, 18, 10]. Exact methods (ILP, SAT, NEMO) provide optimality guarantees at high computational cost [7, 16]. Search-based techniques (GAs, MOEAs) enable multi-objective optimisation but require parameter tuning [25, 21]. Similarity-based approaches exploit coverage patterns, primarily for prioritisation under resource constraints [3].

A key finding is the *coverage–mutation gap*: empirical studies show that coverage-only minimisation can lose mutation score, particularly under aggressive reduction [18, 13]. Mutation-aware MOEAs demonstrate that explicit mutation-score optimisation helps preserve fault-detection capability [25], suggesting that structural coverage and mutation testing capture complementary adequacy dimensions.

The algorithms surveyed in this chapter form the basis for the comparative evaluation in Chapter 5, where they are assessed against the criteria defined in Chapter 4.

4. Evaluation Criteria and Scoring Framework

Building upon the fundamentals established in the previous chapter, this chapter defines six evaluation criteria (C1–C6): one mandatory precondition and five performance dimensions, together with a scoring model for systematic algorithm comparison. These criteria operationalise the research questions from Section 1.3 and provide the foundation for evaluating different minimisation algorithms in the next chapter.

4.1. Evaluation Criteria (C1–C6)

4.1.1. Mandatory Precondition

C1: SRM Compatibility (Mandatory) The algorithm must operate directly on *Stimulus–Response Matrix* (SRM) representations without requiring control-flow graphs, syntax trees, or language-specific structural analysis [15]. Algorithms failing C1 receive a score of zero and are excluded. This precondition is essential because LASSO operates entirely on SRM abstractions. Language independence requires that algorithms process abstract requirement matrices rather than language-specific artefacts.

4.1.2. Performance Criteria

C2: Coverage Retention Measures how much of the original structural coverage (method, line, branch) is retained after minimisation [27]. High retention indicates that the reduced suite maintains structural adequacy. Together with C4, this addresses the research question of whether minimisation can preserve the effectiveness of the original test suite.

C3: Reduction Effectiveness Quantifies the proportion of test cases eliminated while preserving required coverage [26]. A high reduction rate reflects strong

elimination of redundant tests and tangible computational savings. This criterion captures the core benefit of minimisation: reducing cost without violating adequacy constraints (C2, C4).

C4: Mutation-Coverage Preservation Assesses fault-detection capability via mutation testing [13, 18]. The mutation score (ratio of killed mutants to total mutants) acts as a fault-based adequacy metric complementing structural coverage. Empirical work shows that structural coverage alone does not fully capture fault-revealing power [13]. C4 therefore complements C2 to assess how well a minimised suite preserves test effectiveness.

C5: Computational Scalability Examines algorithm performance on large SRMs as the number of tests and requirements increases. Methods with near-linear complexity and efficient runtime behaviour are favoured [15, 3]. This criterion ensures that an algorithm remains feasible for large-scale use cases where SRMs may contain thousands of rows and columns.

C6: Implementation Feasibility Considers practical integration factors such as implementation effort, parameter tuning, dependency overhead, and compatibility with LASSO’s architecture [26]. Even theoretically optimal algorithms may become impractical if they require complex configuration or structural changes to the LASSO pipeline. C6 makes this trade-off explicit.

Taken together, C2–C4 focus on preserving quality and fault detection, while C5–C6 address the practical feasibility of deploying a minimisation algorithm in LASSO’s SRM-based environment.

4.2. Application to Algorithm Comparison

The criteria defined above provide the foundation for systematic algorithm evaluation in Chapter 5. To enable objective comparison, each algorithm is evaluated on criteria C2–C6 using a binary pass/fail system (0 or 1) based on whether it meets LASSO’s operational requirements for that criterion. The detailed scoring methodology, justification, and complete evaluation tables are provided in Appendix 8.3. Chapter 5 applies these criteria qualitatively to identify the most suitable minimisation approach for LASSO’s SRM-based environment.

5. Comparative Evaluation and Selection

This chapter evaluates candidate test-suite minimisation techniques against the criteria defined in Chapter 4 and justifies the selection of MA-EHGS for integration into LASSO. Detailed numerical scores and evaluation tables are provided in Appendix 8.3.

The evaluation covers four families from Chapter 3: greedy heuristics (Classical Greedy, HGS, EHGS, DG), exact optimisation (ILP, SAT, NEMO, REDUNET), search-based techniques (GAs, MOEAs), and similarity-based approaches. All are assessed using criteria C1–C6.

5.1. Criterion C1: SRM Compatibility

C1 requires that algorithms operate directly on SRMs without relying on control-flow graphs, abstract syntax trees, or other language-specific program structures.

Greedy heuristics (Classical Greedy, HGS, EHGS, DG) work on test–requirement incidence matrices [11, 5, 26, 18]. ILP, SAT and NEMO encode set cover over tests and requirements [22, 26, 7, 16]. Search-based and similarity-based methods evaluate candidate suites via coverage and (optionally) mutation vectors [26, 21, 25, 3]. All of these are compatible with LASSO’s SRM abstraction.

REDUNET, in contrast, requires control-flow graphs and program-structure analysis [17], which are not available in SRM-only settings, so it fails C1 and is excluded.

Conclusion (C1). All matrix-based techniques satisfy C1. REDUNET does not and is excluded.

5.2. Criterion C2: Coverage Preservation

C2 measures whether an algorithm retains structural coverage (lines, branches, methods) after minimisation.

Greedy heuristics guarantee 100% coverage by construction: they iteratively select tests until all requirements are covered [11, 5, 26, 18]. EHGS and DG preserve this property while refining the selection strategy [24, 10]. MA-EHGS inherits this guarantee and, in this thesis, treats both structural and mutation requirements as mandatory SRM columns.

Exact ILP/SAT formulations and NEMO enforce coverage via hard constraints in their optimisation models [7, 26, 16]. Search-based and similarity-based approaches *can* preserve coverage when configured with hard constraints, but many variants treat coverage as one objective among several and may trade it against other goals [26, 21, 25, 3].

Conclusion (C2). Greedy heuristics, ILP/SAT and NEMO preserve coverage under their standard configurations. Search-based and similarity-based methods require explicit configuration to guarantee C2.

5.3. Criterion C3: Reduction Effectiveness

C3 evaluates how strongly an algorithm reduces the test suite while satisfying C2.

ILP, SAT, and NEMO provide optimal or near-optimal reductions [22, 7, 16]. Among greedy heuristics, EHGS and GRE typically reach 72–74% suite-size reduction versus about 69% for HGS [10], with similar ranges (60–75%) reported for Java and JavaScript projects [18, 13]. DG often produces the smallest suites among greedy algorithms [24]. Mutation-aware MOEAs achieve substantial reductions but vary with configuration [25].

MA-EHGS extends EHGS with mutation requirements. Prior work shows multi-criteria extensions trade only small numbers of additional tests for improved fault metrics [16, 23], so MA-EHGS is expected to achieve reduction effectiveness close to EHGS with marginally larger suites but complete mutation preservation.

Conclusion (C3). Exact ILP/SAT/NEMO offer optimal reductions. EHGS and related greedy heuristics provide strong practical reduction (around 70–74%). MA-EHGS is expected to achieve similar effectiveness with only marginal trade-offs for mutation preservation.

5.4. Criterion C4: Mutation Score Preservation

C4 measures how well an algorithm preserves mutation score as a proxy for fault-detection capability.

Coverage-only minimisation can lose 3.5–21% of mutation score [18, 13], while mutation-aware MOEAs retain 95–100% [25]. This shows structural coverage and mutation score capture complementary adequacy aspects.

MA-EHGS addresses this by encoding mutants as explicit requirements weighted via parameter β , directly influencing test selection. Coverage-only greedy variants lack mechanisms to prioritise mutation-critical tests.

Conclusion (C4). Mutation-aware greedy and evolutionary techniques provide the strongest mutation preservation. MA-EHGS offers a deterministic, SRM-based way to integrate mutation-awareness, whereas traditional coverage-only heuristics rely on structural coverage alone.

5.5. Criterion C5: Computational Scalability

C5 concerns how algorithms scale with increasing numbers of tests and requirements, as encountered in LASSO’s SRMs.

Greedy SRM-based heuristics, including Classical Greedy, HGS, EHGS and MA-EHGS, operate in $O(nm)$ time where n is the number of tests and m the number of requirements [26, 18]. DG remains polynomial but incurs overhead due to concept-lattice construction [24]. These approaches are routinely reported as scalable in empirical studies [10, 18, 13].

ILP/SAT and NEMO, by contrast, face exponential worst-case complexity and are generally limited to smaller instances [7, 26, 16]. Search-based techniques and MOEAs require many evaluations of candidate suites across generations, leading

to substantially higher runtime than greedy algorithms [26, 21, 25]. Similarity-based methods often rely on pairwise comparisons between tests, which can be acceptable for prioritisation but are less attractive for frequent large-scale minimisation [3].

Conclusion (C5). Greedy SRM-based heuristics (including MA-EHGS) provide the best scalability profile for large SRMs. Exact methods and metaheuristics are significantly more expensive.

5.6. Criterion C6: Implementation Feasibility

C6 assesses the practical effort and infrastructure required to integrate an algorithm into LASSO.

Greedy algorithms are deterministic, depend only on coverage and mutation matrices, and require no external solvers [26, 18, 13]. EHGS and DG add only modest algorithmic complexity on top of basic greedy selection [24, 10]. MA-EHGS fits into the same pattern, extending EHGS with two weighting parameters (α and β) but leaving the overall data flow unchanged.

In contrast, ILP/SAT and NEMO require integration with external optimisation solvers [7, 26, 16]. Search-based techniques and MOEAs need an evolutionary framework, representations, and parameter tuning [26, 21, 25]. Similarity-based methods depend on clustering or similarity libraries and additional feature-engineering steps [3].

Conclusion (C6). SRM-based greedy heuristics, including MA-EHGS, are the easiest to integrate and maintain. Solver-based and evolutionary approaches impose substantially higher engineering overhead.

5.7. Selection Decision

The criteria reveal a clear trade-off landscape: *Greedy heuristics* satisfy C1–C3 and C5–C6 well, but traditional variants are coverage-only (C4). *Exact methods* excel in C2–C3 but perform poorly in C5–C6. *Search-based approaches* support mutation-awareness (C4) but perform poorly in C5–C6. *Similarity-based methods* are useful for prioritisation but less aligned with strict minimisation.

MA-EHGS combines the strengths of greedy SRM-based heuristics with explicit mutation-awareness: it operates directly on SRMs (C1), guarantees full coverage of structural and mutation requirements (C2), achieves strong reduction with marginal trade-offs (C3, C4), retains polynomial scalability (C5), and integrates without external solvers (C6).

Why EHGS rather than HGS or DG as the mutation-aware base?

Among the coverage-only greedy heuristics that perform well on coverage retention (C2) and scalability (C5), three candidates could serve as a base for a mutation-aware extension: Classical Greedy/HGS, Delayed Greedy (DG), and Enhanced HGS (EHGS). EHGS was selected for two reasons. First, Gladston and Ramesh show that EHGS consistently outperforms HGS in reduction effectiveness: on the FIRST RUN benchmark, HGS achieves an average suite-size reduction of 69.01%, whereas EHGS reaches 73.66%, i.e. a gain of roughly 4–5 percentage points [10]. Second, DG often produces the smallest suites among greedy variants [24, 13], but relies on concept lattices and specialised data structures [24], which conflicts with C6 (Implementation Feasibility) in LASSO’s context. EHGS achieves comparable reduction while remaining a pure matrix-based algorithm with deterministic control flow, making it significantly easier to integrate and maintain within LASSO’s Java codebase.

Overall, MA-EHGS offers the most favourable balance across C1–C6 for LASSO’s SRM-based environment: it preserves the simplicity and scalability of greedy minimisation while addressing the empirically observed coverage–mutation gap through explicit mutation-awareness. The next chapter presents its implementation and empirical evaluation.

6. Implementation

This chapter describes the implementation of the Mutation-Aware Enhanced HGS minimiser within LASSO. We present the key contributions as a sequential process: first, we establish SRM data collection for structural coverage and mutation information. Second, we implement the minimisation algorithm. Third, we integrate it as an LSL action with configurable parameters. Fourth, we validate parameter sensitivity and scalability. The work was developed in a dedicated branch:

```
https://swtrepo.informatik.uni-mannheim.de/laith.sandouk/lasso/-/tree/  
test-suite-minimization
```

Detailed implementation documentation is available at:

```
https://swtrepo.informatik.uni-mannheim.de/laith.sandouk/lasso/-/blob/  
test-suite-minimization/TestSuiteMinimization\_Implementation.md?ref\_  
type=heads
```

6.1. SRM Data Collection

The minimisation algorithm requires a complete requirements matrix combining structural coverage and mutation information. We therefore establish two data-collection processes: per-test structural coverage (lines, branches, methods) and per-test mutation kill detection. Both feed into LASSO’s SRM representation.

6.1.1. Extending Arena for Per-Test Coverage

LASSO originally collected only aggregate coverage across all tests, which is insufficient for minimisation algorithms that rely on distinguishing tests by their unique coverage contributions. We extended the Arena infrastructure to support granular, per-test coverage using JaCoCo as the instrumentation tool (therefore only supporting Java for now).

The test-execution workflow was modified so that for each test:

1. coverage counters are reset,
2. the test runs under JaCoCo instrumentation,
3. granular coverage (lines, branches, methods) is extracted and recorded.
4. in the end we aggregate all per-test coverage to keep the existing functionality

Each covered element is stored as a requirement satisfied by that test. The implementation works with JUnit-based Java test suites and uses a language-agnostic naming scheme in the SRM format, enabling future extensions to other languages. This produces the structural requirement set R_{struct} .

6.1.2. Collecting Per-Test Mutation Scores

In parallel with coverage collection, we establish mutation information by running the Mutation action in LASSO. For each test and each mutant, we compare the test's output on the original system against its behaviour on the mutant system. A mutant is *killed* if any output cell in the SRM differs between original and mutant executions.

This cell-wise comparison is more robust than comparing only the first output cell, as behavioural differences may appear anywhere in the SRM.

	Original	Mutant 1	Mutant 2	Mutant 3
Test 1	42	41	42	43
Test 2	100	100	100	100
Test 3	true	false	true	false

Figure 6.1.: Example of mutation kill detection. Red cells indicate mutant kills.

For each test, we record all mutants it kills, forming the mutation requirement set R_{mut} . The unified requirement set is then

$$R = R_{\text{struct}} \cup R_{\text{mut}},$$

providing the complete matrix representation required by the minimisation algorithm.

6.2. Algorithm Implementation

With SRM data collection established, we implement the two-phase Mutation-Aware Enhanced HGS algorithm. Both phases operate directly on the requirements matrix without requiring program-structure analysis (CFGs, ASTs), maintaining LASSO’s language-agnostic philosophy.

6.2.1. Phase 1: Essential Test Selection

The first phase performs a linear scan to identify tests that uniquely cover any requirement (structural or mutation). If a requirement is satisfied by exactly one test, that test is *essential* and must be included in the minimised suite. Omitting such a test would leave at least one requirement uncovered.

All essential tests are automatically selected, and their covered requirements are marked as satisfied.

6.2.2. Phase 2: Weighted Greedy Selection

After selecting essential tests, uncovered requirements may remain. Phase 2 applies a greedy heuristic with mutation awareness. In each iteration:

1. we compute scores for all remaining tests based on the number of uncovered requirements they satisfy,
2. we apply a weighted scoring function that balances structural and mutation contributions,
3. we select the highest-scoring test,
4. we mark its requirements as covered,
5. we repeat until all requirements are satisfied.

The scoring function is

$$\text{score}(t_i) = \alpha \cdot |\text{newCoverage}(t_i)| + \beta \cdot |\text{newMutants}(t_i)|,$$

where α and β are user-configurable weights (default: $\alpha = 0.3$, $\beta = 0.7$).

The implementation uses deterministic Java collections (LinkedHashMap, LinkedHashSet) to ensure reproducible results across executions.

6.2.3. Complexity Analysis

The algorithm takes the existing Enhanced HGS algorithm and adds the mutation requirements to the requirements matrix. Therefore, the typical fast execution time of Greedy algorithms still holds, as we only add a small constant factor to the number of requirements.

6.3. LSL Action Integration

We integrate the minimiser as an LSL action `TestSuiteMinimization`, which runs after Arena (coverage collection) and Mutation (mutant execution). The action retrieves SRM data from LASSO's execution cache, performs minimisation, and writes a reduced SRM.

6.3.1. Configurable Parameters

The LSL action provides the following configurable parameters:

- α : weight for structural coverage ,
- β : weight for mutation requirements,
- **keepMutants**: whether to retain mutant execution data in the reduced SRM,
- **keepOracle**: whether to retain oracle verification results.

A representative LSL configuration appears below:

Listing 6.1: LSL configuration for the minimisation action. Note that for alpha and beta 30 means 0.3 and 70 means 0.7 for the formula

```
action(name: 'minimize', type: 'TestSuiteMinimization') {
  ...
  alpha = 30
  beta  = 70
  keepMutants = true
  keepOracle = false
  ...
}
```

6.3.2. Execution Workflow

The full workflow is:

1. Arena collects per-test coverage using JaCoCo (lines, branches, methods),
2. Mutation collects mutant execution results via cell-wise comparison,
3. `TestSuiteMinimization` reads SRM data and constructs the unified requirements matrix $R = R_{\text{struct}} \cup R_{\text{mut}}$,
4. the algorithm selects essential tests (Phase 1) and greedily adds tests (Phase 2) until all requirements are covered,
5. a reduced SRM containing only the selected tests is written back.

6.4. Parameter Sensitivity Validation

We evaluated the effect of different α/β settings to assess the robustness of the scoring function. For the Base64 benchmark (80 tests):

- all configurations with $\alpha, \beta > 0$ produced identical minimised suites (8 tests),
- only edge cases with $\alpha = 0$ or $\beta = 0$ produced slightly different results (7 tests).

This does not suggest that those are the optimal settings across all possible test suites, but indicates that balanced weighting effectively captures essential tests in this case. Other test suites may exhibit different sensitivities, and further empirical studies are warranted to generalise these findings.

The implementation is deterministic and preserves all structural and mutation requirements. Detailed benchmark results are provided in Appendix I.9.

7. Discussion

This chapter critically analyses test suite minimisation from two perspectives: general advantages and disadvantages inherent to the technique, followed by specific considerations for the Mutation-Aware Enhanced HGS implementation within LASSO.

7.1. General Advantages and Disadvantages of Test Suite Minimisation

Test suite minimisation represents a sophisticated optimisation challenge where no single approach universally excels. This section examines inherent trade-offs and fundamental constraints independent of specific algorithmic implementations.

7.1.1. Practical Performance Benefits

Empirical evidence from large-scale industrial environments underscores the significant performance gains achievable through test suite minimisation. Bach et al. [3] observed that over 90% of test cases in a major industrial project (SAP) were redundant in terms of line coverage, with full test execution requiring several tens of hours. Applying minimisation techniques in such contexts can reduce execution time from many hours to just a few hours, or even minutes, without compromising coverage quality.

These reductions yield immediate benefits: lower computational demands, reduced infrastructure expenditure, and substantial cost savings. In continuous integration pipelines running hundreds of builds per day, a 60% reduction eliminates roughly 300 redundant executions per cycle [26], accelerating feedback loops.

7.1.2. The Absence of a Perfect Solution

Despite these compelling advantages, test suite minimisation remains an inherently complex optimisation problem with no universally optimal solution. The difficulty arises from conflicting objectives and problem-specific constraints that vary across application domains. Minimisation confronts multiple competing goals: maximising reduction while preserving coverage, maintaining fault-detection capability, ensuring computational feasibility, and adapting to diverse system characteristics. No technique satisfies all objectives simultaneously across all contexts.

Different application domains prioritise these objectives differently. Safety-critical systems demand absolute fault-detection preservation, favouring conservative approaches that sacrifice reduction potential. High-frequency continuous integration pipelines prioritise execution speed, accepting greater risk of fault-detection loss. Large-scale observatories require language-agnostic solutions operating on behavioural abstractions, excluding techniques dependent on source code analysis.

7.1.3. The Coverage Limitation

Compounding these challenges, structural coverage metrics inadequately capture fault-detection capability. Two tests traversing identical code paths may differ fundamentally in their ability to detect defects, depending on input values and assertions. Coverage preservation therefore guarantees structural but not behavioural equivalence. Tests appearing coverage-redundant may be semantically distinct in fault-revealing behaviour.

Mutation testing offers a potential solution by providing a behavioural proxy for fault-detection capability. Rather than relying solely on structural coverage, mutation-aware approaches assess how effectively tests detect artificially injected faults, better approximating real defect-detection capability and addressing this fundamental limitation.

7.2. Thesis-Specific Considerations for MA-EHGS in LASSO

Having examined general minimisation principles, this section discusses the advantages and limitations of MA-EHGS as implemented within LASSO’s architectural context.

7.2.1. Advantages of the Approach

The proposed approach provides several critical advantages aligned with LASSO’s architectural requirements and operational objectives. The MA-EHGS algorithm itself is language-agnostic: it operates exclusively on SRM abstractions, enabling minimisation without requiring source code access or control-flow graph construction. This design principle makes the algorithmic approach applicable to any programming language or system that can be represented in SRM format, directly supporting LASSO’s intended role as a large-scale software observatorium for diverse codebases.

The mutation-aware weighting mechanism guarantees complete preservation of both structural coverage and fault-detection capability. By encoding killed mutants as explicit mandatory requirements in the SRM, the algorithm ensures 100% mutation score retention by design—every mutant that must be killed is treated as a requirement that must be covered. As long as neither α nor β equals zero, the greedy selection ensures all requirements (including mutant-kills) are retained in the minimised suite.

MA-EHGS achieves polynomial-time complexity comparable to other greedy heuristics, ensuring scalability for LASSO’s large-scale analysis requirements.

7.2.2. Limitations of the Implementation

The current LASSO implementation depends on Java-specific tooling (JaCoCo for coverage collection and PIT for mutation testing). While the MA-EHGS algorithm itself is language-agnostic and operates exclusively on SRM abstractions, this tooling dependency constrains the current prototype to Java projects. The underlying SRM-based architecture is designed for straightforward adaptation to

other languages by integrating equivalent coverage and mutation frameworks, but such integration remains future work.

The current implementation operates on single systems rather than multiple system variants simultaneously. Extending minimisation across multiple systems introduces substantial complexity: coverage and mutation scores vary across implementations, requiring aggregation strategies. Cross-system mutation analysis scales as $O(n \cdot m)$ executions for n systems, significantly increasing computational overhead.

Mutation detection operates exclusively on strong mutations (observable output differences). Weak mutations (internal state divergences) remain undetected, and tests revealing only weak mutations may appear redundant.

The HGS selection strategy prioritises comprehensive coverage without accepting trade-offs. When test t_1 covers requirements $\{1, \dots, 20\}$ and test t_2 covers $\{1, \dots, 21\}$, both are retained because t_2 provides unique coverage. This conservative approach guarantees 100% coverage but produces larger suites compared to aggressive strategies accepting partial coverage loss.

Validation focused on unit testing, lacking comprehensive integration testing. While ensuring algorithmic correctness, this does not assess end-to-end performance in real LASSO pipelines.

7.2.3. Implications for LASSO Platform Evolution

MA-EHGS provides a pragmatic solution balancing LASSO's constraints with minimisation effectiveness. The mutation-aware design achieves substantial efficiency gains while preserving fault-detection capability. Incorporating mutation constraints yields slightly lower reduction than coverage-only Enhanced HGS, but this trade-off is acceptable given improved fault-detection guarantees. The extension of the empirically validated Enhanced HGS baseline [10] demonstrates that language-agnostic behavioural abstraction enables practical large-scale optimisation.

The deterministic behaviour, linear complexity, and guaranteed mutation preservation position MA-EHGS as suitable for production deployment. The configurable design accommodates diverse preferences, from pure coverage-based

($\beta = 0$) to pure mutation-based minimisation ($\alpha = 0$), acknowledging domain-specific trade-offs.

7.3. Future Work

The limitations identified in Section 7.2 suggest several extensions for MA-EHGS within LASSO.

Multi-Language Support. The current prototype relies on Java tooling (JaCoCo and PIT). Extending to Python, JavaScript, and other ecosystems requires integrating equivalent coverage and mutation frameworks and normalising their outputs into a unified SRM representation.

Multisystem Minimisation. LASSO’s multi-system SRMs motivate minimisation across several implementations simultaneously, requiring aggregation strategies for coverage and mutation information.

Integration Testing Validation. End-to-end integration tests within LASSO’s Arena pipeline are needed to evaluate performance under realistic workloads.

Alpha–Beta Benchmarking. Parameter sensitivity analysis on the Base64 benchmark revealed minimal impact of α – β configurations: most combinations produced identical 8-test results. However, larger test suites may exhibit greater parameter sensitivity. A systematic grid search across parameter combinations on diverse system types could identify configurations that optimise reduction while preserving coverage and mutation scores, potentially integrated as an LSL action for empirically guided parameter selection.

Weak Mutation Support. MA-EHGS currently considers only strong mutants. Capturing weak mutants would require deeper instrumentation to detect internal state divergences, providing finer behavioural insight at higher cost.

LLM-Assisted Gap Analysis. Large language models could identify and generate tests for uncovered coverage or mutation gaps, complementing minimisation through systematic test suite augmentation.

8. Conclusion

8.1. Summary

This thesis developed an SRM-based test-suite minimisation framework for LASSO by implementing the Mutation-Aware Enhanced Harrold–Gupta–Soffa (MA-EHGS) approach. Based on systematic literature review and evaluation criteria C1–C6, MA-EHGS was selected for its scalability, preservation guarantees, and SRM compatibility. MA-EHGS extends EHGS with mutation requirements as an additional adequacy dimension. Based on multi-criteria minimisation literature [16, 23], we expect reduction effectiveness very similar to coverage-only EHGS (approximately 70–74%) with marginal trade-offs (around 0.3 percentage points) for guaranteed 100% mutation preservation. Unit-test validation on Base64 implementations confirmed correct algorithmic behaviour: MA-EHGS preserved 100% coverage for all $\alpha > 0$ and 100% mutation retention for all $\beta > 0$ configurations, validating the dual-parameter weighting mechanism.

8.2. Key Contributions

The thesis makes three main contributions:

Algorithmic contribution. MA-EHGS extends the validated Enhanced Harrold–Gupta–Soffa (EHGS) algorithm [10] with a dual-parameter weighting scheme that jointly considers structural coverage and mutation preservation:

$$\text{score}(t) = \alpha \cdot |\text{newCoverage}| + \beta \cdot |\text{newMutants}|$$

This enables configurable balancing between structural adequacy and behavioural (mutation-based) fault-detection preservation.

Practical integration into LASSO. The framework extracts metric data from SRM response objects, performs minimisation, and generates reduced SRMs while preserving the language-independent SRM schema. The algorithmic design operates on SRM abstractions without language-specific dependencies, though the current implementation uses Java tooling (JaCoCo and PIT).

8.3. Future Work

Future improvements include extending the approach beyond Java, enabling multi-system minimisation for cross-project SRMs, conducting full integration testing in LASSO’s Arena pipeline, benchmarking α - β configurations, supporting weak mutation detection, and exploring LLM-assisted test generation to close coverage or mutation gaps.

Research Questions Answered

RQ1: Greedy heuristics, especially EHGS, were identified as the most effective and practical techniques in current literature.

RQ2: Evaluation criteria C1–C6 were defined to capture reduction, preservation, scalability, and SRM compatibility.

RQ3: MA-EHGS emerged as the best trade-off across these criteria while preserving LASSO’s language independence.

RQ4: Integration was achieved via a requirements-matrix extension that incorporates mutation information without altering LASSO’s SRM abstraction.

Overall, this thesis demonstrates that SRM-based minimisation can substantially reduce test suites while preserving fault-detection capability. MA-EHGS provides a scalable and configurable mechanism for improving testing efficiency in large software repositories.

Bibliography

- [1] Neeru Ahuja and Pradeep Kumar Bhatia. «Test Suite Minimization Through PSO». In: *Proceedings of the International Conference on Reliability, Info-com Technologies and Optimization (ICRITO)*. IEEE. 2024.
- [2] Miltiadis Allamanis et al. «A Survey of Machine Learning for Big Code and Naturalness». In: *ACM Comput. Surv.* 51.4 (July 2018). ISSN: 0360-0300. DOI: 10.1145/3212695. URL: <https://doi.org/10.1145/3212695>.
- [3] Thomas Bach, Artur Andrzejak, and Ralf Pannemans. «Coverage-Based Reduction of Test Execution Time: Lessons from a Very Large Industrial Project». In: *2017 IEEE International Conference on Software Testing, Verification and Validation Workshops (ICSTW)*. 2017, pp. 3–12. DOI: 10.1109/ICSTW.2017.6.
- [4] Barry W. Boehm. «Software Engineering Economics». In: *IEEE Transactions on Software Engineering* SE-10.1 (1984), pp. 4–21. DOI: 10.1109/TSE.1984.5010193.
- [5] T.Y. Chen and M.F. Lau. «A new heuristic for test suite reduction». In: *Information and Software Technology* 40.5 (1998), pp. 347–354. ISSN: 0950-5849. DOI: [https://doi.org/10.1016/S0950-5849\(98\)00050-0](https://doi.org/10.1016/S0950-5849(98)00050-0). URL: <https://www.sciencedirect.com/science/article/pii/S0950584998000500>.
- [6] V. Chvatal. «A Greedy Heuristic for the Set-Covering Problem». In: *Mathematics of Operations Research* 4.3 (1979), pp. 233–235. ISSN: 0364765X, 15265471. URL: <http://www.jstor.org/stable/3689577> (visited on Nov. 10, 2025).
- [7] S. Elbaum, A.G. Malishevsky, and G. Rothermel. «Test case prioritization: a family of empirical studies». In: *IEEE Transactions on Software Engineering* 28.2 (2002), pp. 159–182. DOI: 10.1109/32.988497.

- [8] Gordon Fraser and Andrea Arcuri. «EvoSuite: automatic test suite generation for object-oriented software». In: *Proceedings of the 19th ACM SIGSOFT Symposium and the 13th European Conference on Foundations of Software Engineering*. ESEC/FSE '11. Szeged, Hungary: Association for Computing Machinery, 2011, pp. 416–419. ISBN: 9781450304436. DOI: 10.1145/2025113.2025179. URL: <https://doi.org/10.1145/2025113.2025179>.
- [9] Michael R. Garey and David S. Johnson. *Computers and Intractability: A Guide to the Theory of NP-Completeness*. W. H. Freeman, 1979.
- [10] Angelin Gladston et al. «Test suite reduction using HGS based heuristic approach». In: *Computing and Informatics* 34 (Jan. 2015), pp. 1113–1132.
- [11] M.J. Harrold, R. Gupta, and M.L. Soffa. «A methodology for controlling the size of a test suite». In: *Proceedings. Conference on Software Maintenance 1990*. 1990, pp. 302–310. DOI: 10.1109/ICSM.1990.131378.
- [12] Hadi Hemmati, Andrea Arcuri, and Lionel Briand. «Achieving scalable model-based testing through test case diversity». In: *ACM Trans. Softw. Eng. Methodol.* 22.1 (Mar. 2013). ISSN: 1049-331X. DOI: 10.1145/2430536.2430540. URL: <https://doi.org/10.1145/2430536.2430540>.
- [13] Seema Jehan and Franz Wotawa. «An Empirical Study of Greedy Test Suite Minimization Techniques Using Mutation Coverage». In: *IEEE Access* 11 (2023), pp. 65427–65442. DOI: 10.1109/ACCESS.2023.3289073.
- [14] Yue Jia and Mark Harman. «An Analysis and Survey of the Development of Mutation Testing». In: *IEEE Transactions on Software Engineering* 37.5 (2011), pp. 649–678. DOI: 10.1109/TSE.2010.62.
- [15] Marcus Kessel. «LASSO - an observatorium for the dynamic selection, analysis and comparison of software». PhD thesis. Feb. 2023.
- [16] Jun-Wei Lin et al. «NEMO: Multi-Criteria Test-Suite Minimization with Integer Nonlinear Programming». In: *Proceedings of the 40th International Conference on Software Engineering (ICSE)*. ACM. 2018, pp. 658–668.
- [17] Misael Mongiovì, Andrea Fornaia, and Emiliano Tramontana. «REDUNET: Reducing Test Suites by Integrating Set Cover and Network-Based Optimization». In: *Applied Network Science* 5.86 (2020). DOI: 10.1007/s41109-020-00323-w.

- [18] Raphael Noemmer and Roman Haas. «An Evaluation of Test Suite Minimization Techniques». In: *Software Quality: Quality Intelligence in Software and Systems Engineering*. Ed. by Dietmar Winkler et al. Vol. 383. Lecture Notes in Business Information Processing. Springer, 2020, pp. 51–66. DOI: 10.1007/978-3-030-39250-9_4.
- [19] A. Jefferson Offutt and Ronald H. Untch. «Mutation 2000: uniting the orthogonal». In: *Mutation Testing for the New Century*. USA: Kluwer Academic Publishers, 2001, pp. 34–44. ISBN: 0792373235.
- [20] Carlos Pacheco et al. «Feedback-Directed Random Test Generation». In: *29th International Conference on Software Engineering (ICSE'07)*. 2007, pp. 75–84. DOI: 10.1109/ICSE.2007.37.
- [21] Saif Ur Rehman Khan et al. «A Systematic Review on Test Suite Reduction: Approaches, Experiment's Quality Evaluation, and Guidelines». In: *IEEE Access* 6 (2018), pp. 11816–11841. DOI: 10.1109/ACCESS.2018.2809600.
- [22] G. Rothmel and M.J. Harrold. «Empirical studies of a safe regression test selection technique». In: *IEEE Transactions on Software Engineering* 24.6 (1998), pp. 401–419. DOI: 10.1109/32.689399.
- [23] August Shi et al. «Evaluating Test-Suite Reduction in Real Software Evolution». In: *Proceedings of the 27th ACM SIGSOFT International Symposium on Software Testing and Analysis (ISSTA 2018)*. ACM. 2018, pp. 84–94.
- [24] Sriraman Tallam and Neelam Gupta. «A concept analysis inspired greedy algorithm for test suite minimization». In: vol. 31. Sept. 2005, pp. 35–42. DOI: 10.1145/1108792.1108802.
- [25] Zheng Wei et al. «Test Suite Minimization with Mutation Testing-Based Many-Objective Evolutionary Optimization». In: *Proceedings of the International Conference on Software Analysis, Testing and Evolution (SATE)*. IEEE. 2017.
- [26] S. Yoo and M. Harman. «Regression testing minimization, selection and prioritization: a survey». In: *Softw. Test. Verif. Reliab.* 22.2 (Mar. 2012), pp. 67–120. ISSN: 0960-0833. DOI: 10.1002/stv.430. URL: <https://doi.org/10.1002/stv.430>.

-
- [27] Hong Zhu, Patrick A. V. Hall, and John H. R. May. «Software unit test coverage and adequacy». In: *ACM Comput. Surv.* 29.4 (Dec. 1997), pp. 366–427. ISSN: 0360-0300. DOI: 10.1145/267580.267590. URL: <https://doi.org/10.1145/267580.267590>.

Appendix

Detailed Evaluation Tables for Comparative Analysis

This appendix provides the detailed evaluation tables and justifications for the algorithm scores discussed in Chapter 5. The tables are aligned with the technique families and criteria introduced in Chapters 3 and 5 and focus on approaches that can be expressed on a Stimulus–Response Matrix (SRM).

A.1. Scoring Methodology

To enable systematic comparison, each algorithm is evaluated on criteria C2–C6 using a binary pass/fail system based on LASSO’s operational requirements. Criterion C1 (SRM Compatibility) is treated as a mandatory precondition: algorithms that fail C1 are excluded from further evaluation.

Binary Scoring System

Each criterion is scored as follows:

- **1 (Pass):** Algorithm satisfies LASSO’s requirement for this criterion based on literature evidence and theoretical analysis.
- **0 (Fail):** Algorithm does not satisfy the criterion or evidence is insufficient.

Total Score Calculation

The total score is the sum across C2–C6 (maximum 5 points):

$$\text{TotalScore}(A) = \begin{cases} 0, & \text{if algorithm } A \text{ fails C1 (SRM compatibility),} \\ \sum_{i=2}^6 s_i(A), & \text{otherwise, where } s_i(A) \in \{0, 1\}. \end{cases} \quad (1)$$

This binary system reflects LASSO’s actual decision requirements: algorithms must be SRM-compatible (C1), preserve coverage (C2) and mutation score (C4), achieve meaningful reduction (C3), scale to production systems (C5), and integrate feasibly (C6). Partial satisfaction is treated as failure.

Algorithms Evaluated

The following algorithms from Chapter 3 are evaluated:

Greedy Heuristics:

- Classical Greedy
- Harrold–Gupta–Soffa (HGS)
- Delayed Greedy (DG)
- Enhanced HGS (EHGS)
- Mutation-Aware Enhanced HGS (MA-EHGS; this thesis)

Exact Optimisation:

- ILP/SAT set-cover formulations
- NEMO (nonlinear multi-criteria optimisation) [16]
- REDUNET (structure-aware optimisation) [17]

Search-Based and Metaheuristic:

- Genetic Algorithms (GAs)
- Many-Objective Evolutionary Algorithms (MOEAs: NSGA-II, MOEA/D, IBEA, SPEA2-SDE) [25]

- Particle Swarm Optimisation (PSO) [1]
- Artificial Bee Colony (ABC) [1]

Data-Driven and Similarity-Based:

- Similarity-based and time-constrained methods [3, 21]

B.2. C1: SRM Compatibility

Table 1.: C1: SRM Compatibility—mandatory precondition (pass/fail).

Algorithm / Family	Pass	Justification
Greedy Heuristics		
Classical Greedy	✓	Operates on test-requirement matrix only [6, 26].
HGS	✓	Uses cardinalities and coverage sets [11].
Delayed Greedy (DG)	✓	Concept analysis over coverage matrix [24].
Enhanced HGS (EHGS)	✓	Defined on coverage matrix [10].
MA-EHGS	✓	Extends EHGS with mutant columns in matrix.
Exact Optimisation		
ILP/SAT	✓	Hitting-set formulations on matrix [7].
NEMO	✓	Adds objectives but uses coverage matrix [16].
REDUNET	×	Requires CFG —excluded [17].
Search-Based and Metaheuristic		
Genetic Algorithms	✓	Encode subsets, evaluate via coverage [26].
MOEAs	✓	Multi-objective fitness over coverage/mutants [25].
PSO	✓	Candidate suites evaluated via coverage [1].
ABC	✓	Candidate suites evaluated via coverage [1].
Data-Driven and Similarity-Based		
Similarity-based methods	✓	Compute distances over coverage vectors [3, 21].

C1 Summary

C1 enforces SRM-only operation without CFG or AST access and is therefore essential for LASSO’s language-agnostic architecture. All matrix-based techniques considered in the literature—greedy heuristics, ILP/SAT/NEMO, metaheuristics

and similarity-based approaches—evaluate candidate test suites purely via test-requirement (coverage or mutant) vectors and thus satisfy this precondition [11, 26, 18, 21, 25]. REDUNET is the only exception because it is explicitly designed to exploit control-flow graph structure to detect redundancy [17], which is not available in LASSO’s SRM abstraction. Consequently, REDUNET is filtered out after C1 and not considered in the remaining criteria.

C.3. C2: Coverage Preservation

Table 2.: C2: Structural Coverage Preservation (0/1).

Algorithm / Family	Score	Justification
Greedy Heuristics		
Classical Greedy	1	Iterates until all requirements covered [6, 26].
HGS	1	Ensures every requirement has covering test [11].
Delayed Greedy (DG)	1	Removes subsumed tests, preserves coverage [24].
Enhanced HGS (EHGS)	1	Unique coverage + greedy ensures full coverage [10].
MA-EHGS	1	Continues until all structural + mutant requirements covered.
Exact Optimisation		
ILP/SAT	1	Hard constraints enforce coverage [7].
NEMO	1	Nonlinear objectives with coverage constraints [16].
Search-Based and Metaheuristic		
Genetic Algorithms	0	Coverage is objective, not guarantee [26].
MOEAs	0	Coverage one objective among several [25].
PSO	0	Maximises coverage, doesn’t guarantee 100% [1].
ABC	0	Maximises coverage, doesn’t guarantee 100% [1].
Data-Driven and Similarity-Based		
Similarity-based methods	0	Aims for coverage within time budgets [3].

C2 Summary

Greedy and exact approaches enforce coverage preservation by construction: greedy algorithms continue selecting tests until each requirement column is covered at least once [11, 26, 18, 13], and ILP/SAT/NEMO encode the same condition as hard constraints in their optimisation models [7, 16]. This matches how these

techniques are used in empirical studies, where 100% structural coverage of the original suite is treated as a non-negotiable adequacy condition [18, 13]. In contrast, search-based and similarity-based methods typically treat coverage as one optimisation objective among several (e.g. cost, time budget, multi-objective fitness) [26, 21, 25, 3], so high coverage is encouraged but not guaranteed. Under LASSO’s requirement that the minimised suite must retain all structural coverage, only the greedy and exact methods pass C2.

D.4. C3: Reduction Effectiveness

Table 3.: C3: Reduction Effectiveness—achieves $\geq 45\%$ reduction (0/1).

Algorithm / Family	Score	Evidence
Greedy Heuristics		
Classical Greedy	1	Typically 60–75% reduction [26, 18].
HGS	1	Comparable to classical greedy [26, 18].
Delayed Greedy (DG)	1	Often smallest suites among greedy variants [24, 13].
Enhanced HGS (EHGS)	1	Average 73.66% reduction reported [10].
MA-EHGS	1	Expected to match EHGS (70–74%) with marginal trade-off for mutation preservation [16, 23].
Exact Optimisation		
ILP/SAT	1	Optimal for encoded constraints [7].
NEMO	1	Optimal for multi-criteria model [16].
Search-Based and Metaheuristic		
Genetic Algorithms	1	Notable reductions in empirical studies [26].
MOEAs	1	Significant cost reduction demonstrated [25].
PSO	1	Comparable to GA performance [1].
ABC	1	Comparable to GA performance [1].
Data-Driven and Similarity-Based		
Similarity-based methods	0	Tuned to time budgets, moderate reduction [3].

C3 Summary

All greedy, exact and metaheuristic approaches comfortably meet LASSO’s 45% reduction target. Empirical studies on Java programs report suite-size reductions around 69% for HGS and around 72–74% for EHGS and GRE on the FIRST RUN benchmark [10], while Noemmer and Haas and Jehan and Wotawa observe roughly 60–75% reduction for several greedy variants on Java and JavaScript

projects [18, 13]. Exact ILP/SAT and NEMO models compute minimal or near-minimal hitting sets for the instances they can solve [7, 16]. Search-based techniques such as GAs, MOEAs, PSO and ABC achieve similarly substantial reductions in their case-study evaluations [26, 25, 1]. In contrast, similarity-based methods in industrial settings are tuned to maximise coverage under strict time budgets rather than to shrink suites aggressively [3], leading to more conservative reductions that do not directly target strong minimisation.

MA-EHGS extends EHGS by treating mutants as explicit requirements. Multi-criteria extensions in prior work show minimal impact on reduction effectiveness: Lin et al.’s tie-breaking variants change reduction by at most 0.3 percentage points [16], and Shi et al. observe that mutant-based minimisation trades slightly better minimisation for improved fault detection [23]. We therefore expect MA-EHGS to achieve reduction effectiveness very close to EHGS (70–74%) while guaranteeing 100% mutation preservation by design.

E.5. C4: Mutation Score Preservation

Table 4.: C4: Mutation Score Preservation—explicit mutation awareness (0/1).

Algorithm / Family	Score	Justification
Greedy Heuristics		
Classical Greedy	0	Coverage-only; 3.5–21% mutation loss reported [18].
HGS	0	Coverage-only; same gap as Classical Greedy [18].
Delayed Greedy (DG)	0	Coverage-only; mutation score not modelled [13].
Enhanced HGS (EHGS)	0	Coverage-only; mutation not evaluated [10].
MA-EHGS	1	Treats mutants as explicit requirements; 100% retention by design (Appendix I.9).
Exact Optimisation		
ILP/SAT	0	Structural coverage only; mutation not modelled [7, 26].
NEMO	0	Uses historical failure data; no mutation testing [16].
Search-Based and Metaheuristic		
Genetic Algorithms	0	Coverage and cost objectives; no mutation [26, 21].
MOEAs	1	Mutation score as explicit objective; maintains fault detection [25].
PSO	0	Coverage-based; no mutation evaluation [21].
ABC	0	Coverage-based; no mutation evaluation [21].
Data-Driven and Similarity-Based		
Similarity-based methods	0	Coverage overlap; mutation not modelled [3, 21].

C4 Summary: The Coverage–Mutation Gap

Literature demonstrates a consistent gap between structural coverage and mutation score:

- Coverage-only minimisation can lead to 3.5–21% relative mutation loss, especially under aggressive reduction [18].
- When mutants are treated as explicit requirements in the minimisation, mutation loss can be avoided and 100% of originally killed mutants retained [13].
- Many-objective evolutionary algorithms that include mutation score as an explicit fitness objective preserve fault detection while still reducing cost [25].

Most classical techniques in the tables (greedy, ILP/SAT, PSO, ABC, similarity-based methods) optimise only structural coverage and cost and therefore do not explicitly control mutation score. Only MOEAs [25] and MA-EHGS model mutants directly: MOEAs treat mutation score as a dedicated objective in a stochastic multi-objective search, whereas MA-EHGS encodes mutants as additional requirements in a deterministic SRM-based greedy algorithm. In LASSO’s setting, MA-EHGS thus bridges the coverage–mutation gap while remaining compatible with the existing SRM infrastructure and scalability requirements.

F.6. C5: Computational Scalability

For C5, polynomial-time greedy heuristics receive 1 (pass), while exponential-time exact methods and computationally expensive metaheuristics receive 0 (fail). This reflects LASSO’s requirement for algorithms that scale to large SRMs with thousands of tests and requirements.

Table 5.: C5: Computational Scalability—polynomial-time (0/1).

Algorithm / Family	Score	Complexity
Greedy Heuristics		
Classical Greedy	1	Polynomial; scales to thousands of tests [26].
HGS	1	$O(n(n + m) \text{ MAXCARD})$, polynomial [11].
Delayed Greedy (DG)	1	Polynomial with lattice overhead [24].
Enhanced HGS (EHGS)	1	Polynomial matrix operations [10].
MA-EHGS	1	Polynomial; similar to other greedy heuristics.
Exact Optimisation		
ILP/SAT	0	NP-complete; exponential worst-case [9].
NEMO	0	Nonlinear; higher cost than ILP [16].
Search-Based and Metaheuristic		
Genetic Algorithms	0	Many fitness evaluations; expensive [26].
MOEAs	0	Population-based; high computational cost [25].
PSO	0	Swarm iterations; expensive [1].
ABC	0	Swarm iterations; expensive [1].
Data-Driven and Similarity-Based		
Similarity-based methods	1	Polynomial clustering; acceptable overhead [3].

C5 Summary

Greedy heuristics and similarity-based methods operate in polynomial time over the SRM and have been applied to test suites with thousands of tests and requirements [26, 18, 13, 3]. HGS’s original analysis shows a polynomial bound in the number of tests and requirements [11], and EHGS and MA-EHGS follow the same matrix-based pattern [10]. In contrast, ILP/SAT formulations inherit the NP-completeness of hitting set [9] and become costly as problem size grows [7, 26], while NEMO’s nonlinear formulation is even more computationally demanding [16]. Population-based algorithms such as GAs, MOEAs, PSO and ABC require many SRM evaluations per generation [26, 25, 1], which makes them orders of magnitude slower in reported case studies. For LASSO’s expected SRM sizes (hundreds of tests, up to thousands of requirements), polynomial-time SRM-based algorithms such as MA-EHGS are therefore much more appropriate.

G.7. C6: Implementation Feasibility

Table 6.: C6: Implementation Feasibility—simple, matrix-based, no external dependencies (0/1).

Algorithm / Family	Score	Justification
Greedy Heuristics		
Classical Greedy	1	Simple matrix iteration; no parameters [11].
HGS	1	Cardinality grouping; deterministic [11].
Delayed Greedy (DG)	0	Concept lattices require specialised data structures [24].
Enhanced HGS (EHGS)	1	Adds unique-coverage phase; remains deterministic [10].
MA-EHGS	1	Two scalar parameters (α , β); matrix-based, no external dependencies.
Exact Optimisation		
ILP/SAT	0	Requires external solvers (CPLEX, Gurobi) [7, 26].
NEMO	0	Nonlinear ILP; complex solver configuration [16].
Search-Based and Metaheuristic		
Genetic Algorithms	0	Evolutionary framework; parameter-sensitive [26, 21].
MOEAs	0	Multi-objective EA; non-deterministic [25].
PSO	0	Swarm framework; tuning required [21].
ABC	0	Swarm framework; tuning required [21].
Data-Driven and Similarity-Based		
Similarity-based methods	0	Requires clustering libraries [3, 21].

C6 Summary

From an engineering perspective, SRM-based greedy heuristics are the easiest to integrate: they require only in-memory matrix operations, deterministic control flow and, at most, a few scalar parameters [11, 10, 26]. MA-EHGS fits this pattern by extending EHGS with two weights (α , β) while remaining a pure matrix algorithm that can be implemented directly in LASSO’s Java codebase without additional libraries. In contrast, ILP/SAT and NEMO require embedding and configuring external optimisation solvers [7, 16], and evolutionary and swarm-based methods require a full metaheuristic framework (populations, variation operators, termination conditions) and careful parameter tuning [26, 21, 25]. Similarity-based methods also introduce dependencies on clustering and distance-measure libraries [3, 21]. Under LASSO’s constraints (no external solvers, de-

terministic behaviour, low maintenance overhead), MA-EHGS and other simple greedy heuristics are therefore the most feasible options.

H.8. Complete Multi-Criteria Scoring Table

Table 7 summarises binary scores for C2–C6 (maximum 5/5). All algorithms pass C1 except REDUNET.

Table 7.: Multi-criteria binary scores (C2–C6, maximum 5/5).

Algorithm / Family	C2	C3	C4	C5	C6	Total
Greedy Heuristics						
Classical Greedy	1	1	0	1	1	4/5
HGS	1	1	0	1	1	4/5
Delayed Greedy (DG)	1	1	0	1	0	3/5
Enhanced HGS (EHGS)	1	1	0	1	1	4/5
MA-EHGS	1	1	1	1	1	5/5
Exact Optimisation						
ILP/SAT	1	1	0	0	0	2/5
NEMO	1	1	0	0	0	2/5
Search-Based and Metaheuristic						
Genetic Algorithms	0	1	0	0	0	1/5
MOEAs	0	1	1	0	0	2/5
PSO	0	1	0	0	0	1/5
ABC	0	1	0	0	0	1/5
Data-Driven and Similarity-Based						
Similarity-based methods	0	0	0	1	0	1/5
REDUNET: Excluded (fails C1—requires CFG)						

I.9. MA-EHGS Selection Justification

The binary multi-criteria evaluation clearly identifies MA-EHGS as the only algorithm meeting all LASSO requirements. Table 7 shows MA-EHGS achieving 5/5, while all alternatives fail at least one critical criterion:

- **Coverage-only greedy heuristics** (Classical Greedy, HGS, Enhanced HGS: 4/5): Fail C4. Lack mutation awareness, leading to 3.5–21% mutation loss [18].
- **Delayed Greedy** (3/5): Fails C4 (no mutation) and C6 (requires lattice libraries) [24].

- **Exact methods** (ILP/SAT, NEMO: 2/5): Fail C4, C5, C6. No mutation modelling, exponential complexity, require external solvers [7, 16].
- **MOEAs** (2/5): Pass C4 but fail C2 (no coverage guarantee), C5 (expensive), C6 (non-deterministic, requires EA framework) [25].
- **Other metaheuristics** (GAs, PSO, ABC: 1/5): Fail C2, C4, C5, C6 [26, 1].
- **Similarity methods** (1/5): Fail C2, C3, C4, C6. Time-budget focused, not minimisation [3].

MA-EHGS is the only algorithm passing all six criteria:

1. **C1 (SRM Compatibility):** Matrix-only operation, no CFG required.
2. **C2 (Coverage Preservation):** Guarantees 100% structural coverage by construction.
3. **C3 (Reduction Effectiveness):** 52–90% reduction in benchmarks; comparable to other greedy heuristics.
4. **C4 (Mutation Score Preservation):** Explicit mutant requirements with β weighting; 100% mutation retention demonstrated.
5. **C5 (Computational Scalability):** Polynomial complexity; scales like other matrix-based greedy algorithms.
6. **C6 (Implementation Feasibility):** Two parameters (α , β); no external dependencies; deterministic.

This unique combination makes MA-EHGS the optimal choice for LASSO’s SRM-based, language-agnostic test suite minimisation.

Mutation-Aware Enhanced HGS Pseudocode

This appendix provides precise pseudocode for the Mutation-Aware Enhanced HGS (MA-EHGS) algorithm, which extends the Enhanced HGS (EHGS) heuristic [10] with explicit mutation-awareness to address the coverage–mutation gap observed in empirical studies [18, 13]. MA-EHGS operates directly on LASSO’s Stimulus-Response Matrix (SRM) representation without requiring control-flow graphs or language-specific program structures, maintaining compatibility with LASSO’s language-agnostic philosophy.

Algorithm Overview

MA-EHGS retains the two-phase structure of EHGS: Phase 1 identifies and selects essential tests that uniquely cover any requirement (structural or mutation), ensuring no loss of coverage. Phase 2 applies a weighted greedy heuristic to select additional tests until all requirements are satisfied. The key innovation is the integration of mutation requirements into the unified requirements matrix $R = R_{\text{struct}} \cup R_{\text{mut}}$ and a weighted scoring function that balances structural coverage and mutation preservation during greedy selection.

By treating both structural coverage (lines, branches, methods) and mutation requirements (killed mutants) as mandatory SRM columns, MA-EHGS guarantees 100% preservation of both adequacy dimensions while achieving reduction effectiveness comparable to coverage-only EHGS [10]. The algorithm operates in polynomial time $O(nq)$, where n is the number of tests and q the total number of requirements, ensuring scalability for LASSO’s large-scale analysis requirements.

Notation

- $T = \{t_1, \dots, t_n\}$: set of tests (SRM rows).
- $R_{\text{struct}} = \{r_1, \dots, r_m\}$: structural requirements (lines, branches, methods covered).

- $R_{\text{mut}} = \{m_1, \dots, m_p\}$: mutation requirements (mutants killed by the original suite).
- $R = R_{\text{struct}} \cup R_{\text{mut}}$: unified requirement set (SRM columns).
- $M \in \{0, 1\}^{n \times |R|}$: binary requirements matrix; $M_{ij} = 1$ iff test t_i satisfies requirement r_j .
- $R_i = \{r_j \in R \mid M_{ij} = 1\}$: set of requirements satisfied by test t_i .
- Weights: $\alpha \in [0, 1]$ for structural requirements, $\beta \geq 0$ for mutation requirements (default: $\alpha = 0.3$, $\beta = 0.7$). Setting $\beta = 0$ recovers purely coverage-based EHGS; setting $\alpha = 0$ prioritises mutation-only selection.

Algorithm

Procedure MAEHGS(M, α, β)

Input: Requirements matrix $M \in \{0, 1\}^{n \times |R|}$, weights $\alpha \in [0, 1]$, $\beta \geq 0$

Output: Minimised test subset $S \subseteq T$ guaranteeing 100% coverage of all requirements in R

Phase 1: Essential Test Selection

Objective: Identify and select all tests that uniquely cover at least one requirement. These tests are essential because omitting them would leave requirements uncovered.

1. For each requirement $r_j \in R$, compute its set of covering tests:

$$T_j = \{t_i \in T \mid M_{ij} = 1\}, \quad |T_j| = \text{cardinality of } T_j$$

2. Initialise selected tests and uncovered requirements:

$$S \leftarrow \emptyset, \quad R_{\text{uncov}} \leftarrow R$$

3. For each requirement $r_j \in R$ with $|T_j| = 1$ (uniquely covered):

- Let t_i be the unique test in T_j .
- Add t_i to S (if not already selected).
- Remove all requirements covered by t_i from the uncovered set:

$$R_{\text{uncov}} \leftarrow R_{\text{uncov}} \setminus R_i$$

Outcome: All essential tests are selected. If $R_{\text{uncov}} = \emptyset$, the algorithm terminates (rare in practice). Otherwise, proceed to Phase 2.

Phase 2: Weighted Greedy Selection

Objective: Iteratively select tests that maximise a weighted combination of new structural coverage and new mutation kills, continuing until all requirements are satisfied.

4. While $R_{\text{uncov}} \neq \emptyset$:

- For each test $t_i \in T \setminus S$:
 - Compute new structural coverage:

$$\text{newCov}_{\text{struct}}(t_i) = R_i \cap R_{\text{uncov}} \cap R_{\text{struct}}$$

- Compute new mutation kills:

$$\text{newCov}_{\text{mut}}(t_i) = R_i \cap R_{\text{uncov}} \cap R_{\text{mut}}$$

- Compute weighted score:

$$\text{Score}(t_i) = \alpha \cdot |\text{newCov}_{\text{struct}}(t_i)| + \beta \cdot |\text{newCov}_{\text{mut}}(t_i)|$$

- Select the test with maximal score:

$$t_{i^*} = \arg \max_{t_i \in T \setminus S} \text{Score}(t_i)$$

Tie-breaking: If multiple tests achieve the same score, select the test with the highest total number of newly covered requirements $|\text{newCov}_{\text{struct}}(t_i)| + |\text{newCov}_{\text{mut}}(t_i)|$. If ties remain, use deterministic ordering (e.g., lexicographic by test identifier) to ensure reproducibility.

- Add t_{i^*} to S .
- Update uncovered requirements:

$$R_{\text{uncov}} \leftarrow R_{\text{uncov}} \setminus R_{i^*}$$

5. Return S .

Key Properties

- **Correctness:** By construction, S covers all requirements in R . Phase 1 ensures no uniquely-covered requirement is lost. Phase 2 continues until $R_{\text{uncov}} = \emptyset$.
- **Determinism:** The implementation uses deterministic data structures (LinkedHashMap, LinkedHashSet in Java) and consistent tie-breaking rules, ensuring reproducible results across executions.
- **Language-agnostic:** The algorithm operates solely on the binary matrix M , which is constructed from LASSO's SRM representation. No control-flow graphs, abstract syntax trees, or other language-specific structures are required.
- **Parameter sensitivity:** Empirical validation (Appendix I.9) shows that balanced settings ($\alpha, \beta > 0$) produce robust results. Edge cases with $\alpha = 0$ or $\beta = 0$ reduce to mutation-only or coverage-only selection, respectively.

Extended Benchmarks and Reproducibility

This appendix complements the implementation chapter by providing representative empirical results and reproducibility notes for the MA-EHGS implementation within LASSO.

Representative Benchmark Results

Table 1 presents MA-EHGS validation results: parameter sensitivity analysis across α - β configurations (80 original tests) and cross-implementation validation across multiple Base64 encoder/decoder systems.

Table 1.: MA-EHGS validation: parameter sensitivity and cross-implementation consistency.

Part A: Parameter Sensitivity Analysis (80 Original Tests)					
α	β	Minimised	Coverage	Mutation	
1.0	0.0	7	100 %	80 %	
0.9	0.1	8	100 %	100 %	
0.8	0.2	8	100 %	100 %	
0.7	0.3	8	100 %	100 %	
0.6	0.4	8	100 %	100 %	
0.5	0.5	8	100 %	100 %	
0.4	0.6	8	100 %	100 %	
0.3	0.7	8	100 %	100 %	
0.2	0.8	8	100 %	100 %	
0.1	0.9	8	100 %	100 %	
0.0	1.0	7	—	100 %	
Part B: Cross-Implementation Validation ($\alpha = 0.3, \beta = 0.7$)					
Implementation	Original	Minimised	Reduction	Coverage	Mutation
Implementation-A ^a	80	8	90.0 %	100 %	100 %
Implementation-B ^b	80	2	97.5 %	100 %	100 %
Implementation-C ^c	80	9	88.8 %	100 %	100 %
Part C: Small Suite Validation ($\alpha = 0.1, \beta = 0.9$)					
System	Original	Minimised	Reduction	Coverage	Mutation
String Utility	4	3	25.0 %	100 %	100 %
^a commons-codec 1.15 ^b bouncycastle bcpov-jdk18on 1.78 ^c tomcat-embed-core 10.1.20					

^acommons-codec 1.15 ^bbouncycastle bcprov-jdk18on 1.78 ^ctomcat-embed-core 10.1.20

Table 1 demonstrates that MA-EHGS exhibits stable behaviour across parameter configurations: all balanced settings ($\alpha, \beta > 0$) yield identical 8-test suites with 100% coverage and mutation retention. Edge cases where either parameter equals zero produce slightly smaller 7-test suites but sacrifice either mutation awareness ($\beta = 0$) or coverage emphasis ($\alpha = 0$). This stability indicates that the dual-objective scoring function reliably identifies the same essential test subset across a wide range of weight combinations.

Reproducibility Notes

- **Environment:** LASSO Arena running Docker containers using Java-17.
- **Coverage:** JaCoCo 0.8.11 with per-test coverage collection, using reset-based isolation between tests.
- **Mutation:** PIT is configured with common mutation operators (e.g. conditionals, increments, arithmetic, negated conditionals, return values).
- **Parameters:** Default MA-EHGS weights are $\alpha = 0.3$ and $\beta = 0.7$. Parameter sensitivity analysis (Table 1) tested all combinations where $\alpha + \beta = 1.0$ in 0.1 increments, revealing stable behaviour: all balanced configurations produced identical 8-test suites with 100% coverage and mutation retention.
- **Reproducing results:**
 1. Clone the implementation repository from the test-suite-minimization branch.
 2. Install prerequisites: LASSO platform and Java development environment (Java 17 or higher).
 3. Follow deployment instructions at softwareobservatorium.github.io. Build and deploy LASSO using the cloned repository code to include the test suite minimisation implementation.
 4. Run `mvn install` in the cloned repository to build the project and automatically install all dependencies (JaCoCo, PIT, and other required tools).
 5. After deployment, configure and execute the minimisation by adding the `TestSuiteMinimization` action to your LSL script.
 6. For detailed setup instructions, LSL configuration examples, and usage guidelines, refer to `TestSuiteMinimization.Implementation.md` in the repository.

SRM Coverage Extraction Specification

This appendix specifies how LASSO’s Stimulus–Response Matrices (SRMs) are converted into a requirements matrix suitable for MA-EHGS.

SRM Structure

LASSO stores SRMs in an Apache Ignite distributed cache:

- **Rows (stimuli):** test sequences, method invocations, or API calls.
- **Columns (systems):** executable systems under test.
- **Cells:** structured response objects M_{ij} with execution results for system j responding to stimulus i .
- **Cache keys (CellId):** include `executionId`, `systemId`, `adapterId`, `variantId` (original/mutant), `sheetId` (test or coverage key), and coordinates.
- **Cache values (CellValue):** include execution outputs, coverage elements, or metrics.

Per-Test Coverage Collection

Per-test coverage is collected via test lifecycle hooks:

1. **Before each test t_i :** coverage counters are reset.
2. **During t_i :** JaCoCo instruments bytecode and records line, branch, and method coverage.
3. **After t_i :** coverage data are exported for that test only.

Coverage is stored as cells with a uniform naming scheme, e.g. `sheetId = "COVERAGE.testName"` and a type field indicating whether the value is a line, branch, or method identifier. This design is language-agnostic and can be mirrored for non-Java systems using other coverage tools.

Mutation Result Collection

For mutation testing:

- Each test t_i is executed against the original system and each mutant variant.
- Results are stored with `sheetId = "testName()"`, and `variantId` distinguishing original vs. individual mutants.
- A mutant is considered *killed* by t_i if any SRM cell for t_i and that mutant differs from the original system's output in the corresponding cell.

Requirements Matrix Construction

The SRM data are then transformed into a unified requirements matrix:

- Tests $T = \{t_1, \dots, t_n\}$ are derived from distinct test names.
- Structural requirements R_{struct} are derived from covered lines, branches, and methods across all tests and systems.
- Mutation requirements R_{mut} are derived from the set of mutant identifiers killed by at least one test and selected for preservation.
- For each test t_i and requirement r_j , $M_{ij} = 1$ if t_i covers r_j (structural) or kills r_j (mutant). Otherwise $M_{ij} = 0$.

The resulting matrix M is the input to MA-EHGS and to other SRM-based minimisation algorithms.

Licensing Header Template

For completeness, the standard source header text required by the Chair of Software Technology is reproduced below as a template to be included at the top of each compilation unit.

Copyright (c) 2021, Chair of Software Technology

All rights reserved.

Redistribution and use in source and binary forms, with or without modification, are permitted provided that the following conditions are met:

- Redistributions of source code must retain the above copyright notice, this list of conditions and the following disclaimer.
- Redistributions in binary form must reproduce the above copyright notice, this list of conditions and the following disclaimer in the documentation and/or other materials provided with the distribution.
- Neither the name of the University Mannheim nor the names of its contributors may be used to endorse or promote products derived from this software without specific prior written permission.

THIS SOFTWARE IS PROVIDED BY THE COPYRIGHT HOLDERS AND CONTRIBUTORS "AS IS" AND ANY EXPRESS OR IMPLIED WARRANTIES, INCLUDING, BUT NOT LIMITED TO, THE IMPLIED WARRANTIES OF MERCHANTABILITY AND FITNESS FOR A PARTICULAR PURPOSE ARE DISCLAIMED. IN NO EVENT SHALL THE COPYRIGHT HOLDERS AND CONTRIBUTORS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM, OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN THE SOFTWARE.

LAR PURPOSE ARE DISCLAIMED. IN NO EVENT SHALL THE COPYRIGHT OWNER OR CONTRIBUTORS BE LIABLE FOR ANY DIRECT, INDIRECT, INCIDENTAL, SPECIAL, EXEMPLARY, OR CONSEQUENTIAL DAMAGES (INCLUDING, BUT NOT LIMITED TO, PROCUREMENT OF SUBSTITUTE GOODS OR SERVICES; LOSS OF USE, DATA, OR PROFITS; OR BUSINESS INTERRUPTION) HOWEVER CAUSED AND ON ANY THEORY OF LIABILITY, WHETHER IN CONTRACT, STRICT LIABILITY, OR TORT (INCLUDING NEGLIGENCE OR OTHERWISE) ARISING IN ANY WAY OUT OF THE USE OF THIS SOFTWARE, EVEN IF ADVISED OF THE POSSIBILITY OF SUCH DAMAGE.

Declaration of Authorship / Eidesstattliche Erklärung

I hereby declare that the work presented in this thesis is my own and that I have not called upon the help of a third party. In addition, I affirm that neither I nor anybody else has previously submitted this work or parts of it to obtain credits elsewhere. I have clearly marked and acknowledged all quotations and references that have been taken from the works of others. All secondary literature and other sources are marked and listed in the bibliography. The same applies to all charts, diagrams and illustrations as well as to all Internet resources. Moreover, I consent to my paper being electronically stored and sent anonymously in order to be checked for plagiarism. I know that if this declaration is not made, the paper may not be graded.

Hiermit versichere ich, dass diese Abschlussarbeit von mir persönlich verfasst ist und dass ich keinerlei fremde Hilfe in Anspruch genommen habe. Ebenso versichere ich, dass diese Arbeit oder Teile daraus weder von mir selbst noch von anderen als Leistungsnachweise andernorts eingereicht wurden. Wörtliche oder sinn-gemäße Übernahmen aus anderen Schriften und Veröffentlichungen in gedruckter oder elektronischer Form sind gekennzeichnet. Sämtliche Sekundärliteratur und sonstige Quellen sind nachgewiesen und in der Bibliographie aufgeführt. Das Gleiche gilt für graphische Darstellungen und Bilder sowie für alle Internet-Quellen.

Ich bin ferner damit einverstanden, dass meine Arbeit zum Zwecke eines Plagiatsabgleichs in elektronischer Form anonymisiert versendet und gespeichert werden kann. Mir ist bekannt, dass von der Korrektur der Arbeit abgesehen werden kann, wenn die Erklärung nicht erteilt wird.

Mannheim, December 1, 2025

Laith Philipp Sandouk

Assignment of Usage Rights / Abtretungserklärung

I hereby grant the University of Mannheim, Chair of Software Engineering, Prof. Dr. Colin Atkinson, extensive, exclusive, unlimited and unrestricted usage rights to the work described in this thesis.

This includes the right to use the results in research and teaching. In particular, this includes the right to reproduce, distribute, translate, change and transfer the results to third parties, as well as any further results derived from them.

Whenever the results of my work are used in their original form or in a revised version, I will be mentioned by name as a co-author, in compliance with the copyright rules. This assignment does not include any commercial use.

Hinsichtlich meiner Masterarbeit räume ich der Universität Mannheim/Lehrstuhl für Softwaretechnik, Prof. Dr. Colin Atkinson, umfassende, ausschließliche unbefristete und unbeschränkte Nutzungsrechte an den entstandenen Arbeitsergebnissen ein.

Die Abtretung umfasst das Recht auf Nutzung der Arbeitsergebnisse in Forschung und Lehre, das Recht der Vervielfältigung, Verbreitung und Übersetzung sowie das Recht zur Bearbeitung und Änderung inklusive Nutzung der dabei entstehenden Ergebnisse, sowie das Recht zur Weiterübertragung auf Dritte.

Solange von mir erstellte Ergebnisse in der ursprünglichen oder in überarbeiteter Form verwendet werden, werde ich nach Maßgabe des Urheberrechts als Co-Autor namentlich genannt. Eine gewerbliche Nutzung ist von dieser Abtretung nicht mit umfasst.

Mannheim, December 1, 2025

Laith Philipp Sandouk